Cartesian Trees and Range Minimum Queries

Αναστασοπούλου Αναστασία Παναγιώτα

Διπλωματική Εργασία

Επιβλέπων: Γεωργιάδης Λουκάς

Ιωάννινα, Οκτώβριος, 2026



ΤΜΗΜΑ ΜΗΧ. Η/Υ & ΠΛΗΡΟΦΟΡΙΚΗΣ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
UNIVERSITY OF IOANNINA

Περίληψη

Η παρούσα διπλωματική εργασία έχει ως αντικείμενο τη μελέτη προβλημάτων ερωτημάτων ελαχίστου σε πίνακες και δέντρα, με έμφαση στα προβλήματα Range Minimum Query (RMQ), Lowest Common Ancestor (LCA) και Bottleneck Edge Query (BEQ). Κεντρικό ρόλο στη μελέτη αυτή έχουν οι δομές Cartesian Tree και Cartesian Tree of a Tree, οι οποίες επιτρέπουν την αναγωγή των παραπάνω προβλημάτων μεταξύ τους και την αποδοτική απάντηση των αντίστοιχων ερωτημάτων.

Στο πλαίσιο της εργασίας μελετήθηκαν και υλοποιήθηκαν διαφορετικοί αλγόριθμοι για την επίλυση των παραπάνω προβλημάτων. Για το RMQ εξετάστηκαν τόσο απλές μέθοδοι αναζήτησης όσο και η δομή Sparse Table, καθώς και η αναγωγή του RMQ σε LCA μέσω Cartesian Tree. Για το LCA μελετήθηκαν τόσο online όσο και offline προσεγγίσεις, συμπεριλαμβανομένων τεχνικών που βασίζονται σε Euler tour και ειδικής μορφής RMQ, καθώς και του Tarjan Offline LCA με χρήση Disjoint Set Union. Για το BEQ μελετήθηκε η γενίκευση του Cartesian Tree σε δέντρα μέσω του Cartesian Tree of a Tree, ώστε τα ερωτήματα BEQ να ανάγονται σε ερωτήματα LCA πάνω στη συγκεκριμένη δομή.

Η θεωρητική μελέτη και οι υλοποιήσεις βασίστηκαν κυρίως στην εργασία των Demaine, Landau και Weimann, On Cartesian Trees and Range Minimum Queries, όπου παρουσιάζονται η σχέση μεταξύ RMQ και LCA, η σημασία του Cartesian Tree, καθώς και η γενίκευσή του για το πρόβλημα BEQ. Για τη μελέτη και υλοποίηση των βέλτιστων τεχνικών LCA χρησιμοποιήθηκε επίσης η εργασία των Bender, Farach-Colton, Pemmasani, Skiena και Sumazin, Lowest Common Ancestors in Trees and Directed Acyclic Graphs.

Οι αλγόριθμοι υλοποιήθηκαν σε Java και αξιολογήθηκαν συγκριτικά σε διαφορετικά σύνολα δοκιμών, με στόχο τόσο την επαλήθευση της ορθότητας όσο και τη σύγκριση των διαφορετικών προσεγγίσεων.

Πίνακας Περιεχομένων

Κεφάλαιο 1. Εισαγωγή 1

1.1 Αντικείμενο Εργασίας 1

1.2 Προβλήματα και Εφαρμογές 2

1.3 Περιγραφή της Υλοποίησης 3

Κεφάλαιο 2. Ορισμοί και Θεωρητικό Πλαίσιο 6

2.1 Βασικές Έννοιες Δέντρων 6

2.2 Το Πρόβλημα Range Minimum Query (RMQ) 7

2.3 Το Πρόβλημα Lowest Common Ancestor (LCA) 10

2.4 Το Πρόβλημα Bottleneck Edge Query (BEQ) 12

Κεφάλαιο 3. Cartesian Tree 15

3.1 Ορισμός και Ιδιότητες 15

3.2 Κατασκευή 16

3.3 Σύνδεση RMQ και LCA 19

Κεφάλαιο 4. Sparse Table 21

4.1 Εισαγωγή και Βασική Ιδέα 21

4.2 Κατασκευή 22

4.3 Απάντηση Ερωτημάτων 24

4.4 Προϋπολογισμός Λογαρίθμων 25

Κεφάλαιο 5. Cartesian Tree of Tree 27

5.1 Εισαγωγή και Βασική Ιδέα 27

5.2 Ορισμός και Ιδιότητες 28

5.3 Κατασκευή 29

5.4 Απάντηση BEQ Ερωτημάτων 32

Κεφάλαιο 6. Πειραματική Μελέτη **37**

6.1 Περιγραφή Υλοποίησης 37

6.2 Test Cases για RMQ 40

6.3 Test Cases για BEQ 41

6.4 Αποτελέσματα RMQ 43

6.5 Αποτελέσματα BEQ 45

6.6 Πειράματα σε τυχαίους πίνακες μεγάλης κλίμακας 46

6.7 Πειράματα σε τυχαία δέντρα μεγάλης κλίμακας 50

6.8 Σύνοψη πειραματικών αποτελεσμάτων 53

Κεφάλαιο 7. Συμπεράσματα **55**

7.1 Σύνοψη Αποτελεσμάτων 55

7.2 Συμπεράσματα 56

7.3 Προτάσεις για Περαιτέρω Έρευνα 57

Βιβλιογραφία. **59**

Κεφάλαιο 1. Εισαγωγή

1.1 Αντικείμενο Εργασίας

Η παρούσα διπλωματική εργασία πραγματεύτεται τη μελέτη και υλοποίηση δομών δεδομένων για την αποδοτική απάντηση ερωτημάτων εύρους σε πίνακες και δέντρα. Πιο συγκεκριμένα εστιάζουμε σε τρία θεμελιώδη προβλήματα που αφορούν την αποδοτική αναζήτηση σε δομές δεδομένων.

Το πρώτο πρόβλημα είναι το Range Minimum Query (RMQ), στο οποίο δίνεται ένας πίνακας A από n αριθμούς. Ζητείται η προεπεξεργασία του πίνακα ώστε να μπορούμε να απαντάμε αποδοτικά σε ερωτήματα της μορφής RMQ(i,j)=min{A[i], … ,A[j]} για οποιοδήποτε ζεύγος δεικτών $i \leq j$.

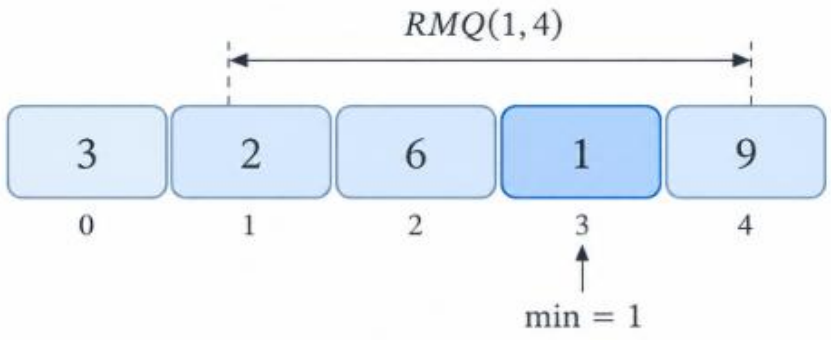
Το δεύτερο πρόβλημα είναι το Lowest Common Ancestor (LCA) και αφορά ριζωμένα δέντρα. Για δύο κόμβους u και v, ζητείται ο βαθύτερος κόμβος του δέντρου που είναι πρόγονος και των δύο. Το LCA συνδέεται άμεσα με το RMQ, καθώς μέσω της κατασκευής του Cartesian Tree ένα ερώτημα RMQ μπορεί να μετατραπεί σε ερώτημα LCA.

Το τρίτο και τελευταίο πρόβλημα είναι το Bottleneck Edge Query (BEQ), το οποίο αποτελεί γενίκευση της ιδέας του RMQ σε σταθμισμένα γραφήματα. Στην παρούσα εργασία εξετάζεται ειδικότερα η περίπτωση όπου το υποκείμενο γράφημα είναι δέντρο. Μεταξύ δύο κόμβων u και v υπάρχει τότε μοναδικό μονοπάτι και το BEQ επιστρέφει την ακμή με το ελάχιστο βάρος πάνω σε αυτό.

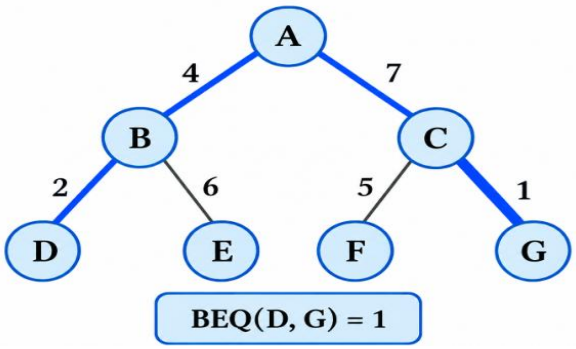
Κεντρικό ρόλο στην επίλυση των παραπάνω προβλημάτων έχουν το Cartesian Tree και η γενίκευσή του σε δέντρα, το Cartesian Tree of a Tree, σε συνδυασμό με τεχνικές για την επίλυση του προβλήματος LCA.

1.2 Προβλήματα και Εφαρμογές

Τα προβλήματα RMQ και BEQ εμφανίζονται σε πλήθος εφαρμογών της Πληροφορικής. Το RMQ χρησιμοποιείται ευρέως σε προβλήματα επεξεργασίας συμβολοσειρών, ενώ το BEQ σχετίζεται άμεσα με προβλήματα ροής δικτύων, όπου η δυνατότητα μεταφοράς μεταξύ δύο κόμβων κατά μήκος ενός μονοπατιού περιορίζεται από την ακμή με το ελάχιστο βάρος.

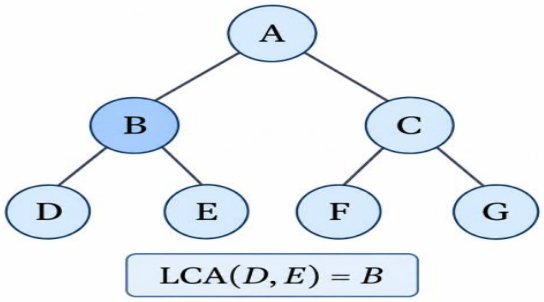


Εικόνα 1.1. Παράδειγμα ερωτήματος RMQ σε πίνακα

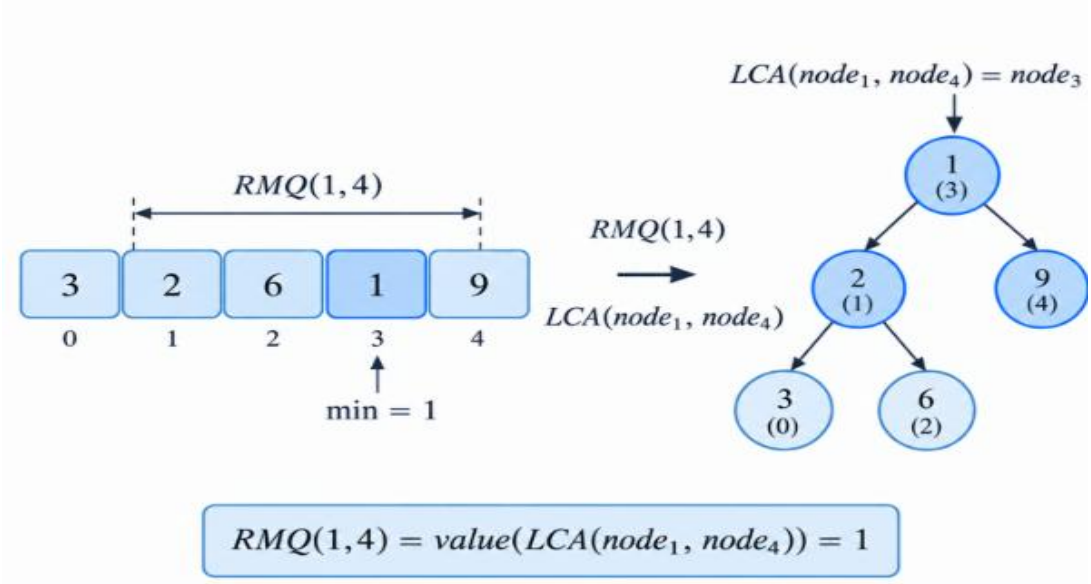


Εικόνα 1.2. Παράδειγμα ερωτήματος BEQ σε σταθμισμένο δέντρο

Ιδιαίτερη σημασία έχει η σχέση του RMQ με το πρόβλημα Lowest Common Ancestor (LCA), καθώς τα δύο προβλήματα συνδέονται μέσω αναγωγών γραμμικού χρόνου. Συγκεκριμένα, ένα ερώτημα RMQ(i,j) στον πίνακα A μεταφράζεται σε ερώτημα LCA($node_i$,$node_j$) στο Cartesian Tree του A, ενώ αντίστροφα το LCA μπορεί να αναχθεί σε RMQ μέσω Euler tour του δέντρου και του αντίστοιχου πίνακα βαθών.



Εικόνα 1.3. Παράδειγμα Lowest Common Ancestor (LCA) σε ριζωμένο δέντρο



Εικόνα 1.4. Αναγωγή ενός ερωτήματος RMQ σε πρόβλημα LCA μέσω του Cartesian Tree

Η σημασία της αποδοτικής επίλυσης αυτών των προβλημάτων γίνεται εμφανής όταν ο αριθμός των ερωτημάτων είναι πολύ μεγάλος. Μία απλοϊκή προσέγγιση που εξετάζει γραμμικά όλα τα στοιχεία για κάθε ερώτημα απαιτεί χρόνο O(n) ανά ερώτημα και επομένως δεν είναι αποδοτική για μεγάλα σύνολα δεδομένων. Αντίθετα, κατάλληλες δομές δεδομένων μπορούν να μειώσουν σημαντικά τον χρόνο απάντησης. Για παράδειγμα, το Sparse Table που χρησιμοποιείται στην εργασία επιτρέπει την απάντηση ενός ερωτήματος RMQ σε χρόνο O(1) μετά από προεπεξεργασία O(n log n).

Στην εργασία εξετάζονται τόσο online όσο και offline τρόποι επεξεργασίας των ερωτημάτων, ανάλογα με το αν τα ερωτήματα γίνονται γνωστά ένα προς ένα ή είναι διαθέσιμα όλα εκ των προτέρων.

1.3 Περιγραφή της Υλοποίησης

Στην παρούσα εργασία υλοποιούνται στη γλώσσα προγραμματισμού Java διαφορετικές προσεγγίσεις για την επίλυση των προβλημάτων Range Minimum Query (RMQ), Lowest Common Ancestor (LCA) και Bottleneck Edge Query (BEQ). Παράλληλα διατηρούνται απλούστερες μέθοδοι αναφοράς, ώστε να είναι δυνατή η σύγκριση και η επαλήθευση της ορθότητας των αποτελεσμάτων.

Για το πρόβλημα RMQ υλοποιείται αρχικά η απλοϊκή μέθοδος NaiveRMQ, η οποία εξετάζει διαδοχικά όλα τα στοιχεία του ζητούμενου διαστήματος και επιστρέφει το ελάχιστο. Η μέθοδος αυτή απαιτεί χρόνο O(R-L+1) ανά ερώτημα και στη χειρότερη περίπτωση O(n), και χρησιμοποιείται κυρίως ως λύση αναφοράς για την επαλήθευση της ορθότητας των πιο σύνθετων υλοποιήσεων.

Υλοποιείται επίσης η δομή Sparse Table, η οποία βασίζεται στον προϋπολογισμό των ελάχιστων τιμών για διαστήματα μήκους δύναμης του δύο. Η κατασκευή της απαιτεί O(n log n) χρόνο και χώρο, ενώ κάθε ερώτημα RMQ απαντάται σε O(1) χρόνο.

Μία ακόμη προσέγγιση βασίζεται στην κατασκευή του Cartesian Tree του πίνακα και στην αναγωγή κάθε ερωτήματος RMQ σε ερώτημα LCA μεταξύ των κόμβων που αντιστοιχούν στα άκρα του διαστήματος. Το Cartesian Tree κατασκευάζεται με χρήση στοίβας, ώστε κάθε στοιχείο να εισάγεται και να αφαιρείται από αυτή το πολύ μία φορά. Με τον τρόπο αυτό η κατασκευή πραγματοποιείται σε O(n) χρόνο και O(n) βοηθητικό χώρο.

Για την επίλυση του προβλήματος LCA διατηρείται αρχικά μία απλή μέθοδος, η οποία βασίζεται στο βάθος των κόμβων και στους δείκτες προς τους γονείς τους. Επιπλέον υλοποιείται μία προσέγγιση βασισμένη στη βέλτιστη static-preprocessed μέθοδο LCA, η οποία χρησιμοποιεί Euler tour και αναγωγή σε ειδικής μορφής ±1 RMQ. Η θεωρητική μέθοδος επιτυγχάνει O(n) προεπεξεργασία και O(1) χρόνο ανά ερώτημα.

Παράλληλα υλοποιείται και ο Tarjan Offline LCA, για την περίπτωση όπου όλα τα ερωτήματα είναι γνωστά εκ των προτέρων. Ο αλγόριθμος επεξεργάζεται τα ερωτήματα κατά τη διάρκεια διάσχισης DFS και χρησιμοποιεί δομή Disjoint Set Union (Union-Find) με path compression και union by rank. Για n κόμβους και q προκαθορισμένα ερωτήματα, ο συνολικός χρόνος είναι O((n+q)α(n)). Η offline αυτή προσέγγιση εφαρμόζεται και στα RMQ ερωτήματα, αφού πρώτα μετατραπούν σε LCA ερωτήματα πάνω στο Cartesian Tree.

Για το πρόβλημα BEQ χρησιμοποιείται το Cartesian Tree of a Tree (CToT). Στην υλοποίηση της εργασίας οι ακμές του αρχικού δέντρου ταξινομούνται κατά φθίνουσα σειρά βάρους και χρησιμοποιείται Union-Find για τη διατήρηση και συγχώνευση των συνδεδεμένων συνιστωσών. Για κάθε ακμή δημιουργείται ο αντίστοιχος εσωτερικός κόμβος του CToT, ενώ οι αρχικοί κόμβοι του δέντρου αντιστοιχούν στα φύλλα της δομής. Για δέντρο με n κορυφές, η συγκεκριμένη κατασκευή απαιτεί O(n log n) χρόνο, με την ταξινόμηση των ακμών να αποτελεί το κυρίαρχο κόστος.

Ένα ερώτημα BEQ(u,v) μετατρέπεται στη συνέχεια σε ερώτημα LCA μεταξύ των φύλλων του CToT που αντιστοιχούν στους κόμβους u και v. Η ακμή που αποθηκεύεται στον αντίστοιχο εσωτερικό κόμβο LCA είναι η bottleneck ακμή του μοναδικού μονοπατιού μεταξύ των δύο αρχικών κόμβων. Εκτός από την υπάρχουσα TreeQuery/LCA προσέγγιση, υλοποιείται και εδώ Tarjan Offline LCA, ώστε ένα σύνολο προκαθορισμένων BEQ ερωτημάτων να μπορεί να επεξεργαστεί συνολικά.

Τέλος, για την επαλήθευση της ορθότητας των υλοποιήσεων πραγματοποιούνται συγκρίσεις πάνω στα ίδια σύνολα δοκιμών. Για το RMQ συγκρίνονται οι NaiveRMQ, Sparse Table, η απλή LCA προσέγγιση, η βέλτιστη LCA προσέγγιση και ο Tarjan Offline LCA, ενώ για το BEQ συγκρίνονται οι NaiveBEQ, TreeQuery/LCA και Tarjan Offline LCA. Τα test cases περιλαμβάνουν τόσο ελεγχόμενες περιπτώσεις όσο και μεγαλύτερα τυχαία παραγόμενα σύνολα δεδομένων.

Κεφάλαιο 2. Ορισμοί και Θεωρητικό Πλαίσιο

Στο κεφάλαιο αυτό παρουσιάζουμε τις βασικές έννοιες και τους ορισμούς που αποτελούν το θεωρητικό υπόβαθρο της εργασίας. Αρχικά ορίζουμε το πρόβλημα Range Minimum Query (RMQ) και εξετάζουμε τη διάκριση μεταξύ της online και της offline εκδοχής του. Στη συνέχεια παρουσιάζουμε το πρόβλημα Lowest Common Ancestor (LCA) και αναδεικνύουμε τη σημαντική σχέση του με το RMQ. Τέλος ορίζουμε το πρόβλημα Bottleneck Edge Query (BEQ) και εξετάζουμε τη σύνδεσή του με τα προηγούμενα προβλήματα.

2.1 Βασικές Έννοιες Δέντρων

Στην ενότητα αυτή παρουσιάζουμε τις βασικές έννοιες της θεωρίας δέντρων που χρησιμοποιούνται στα επόμενα κεφάλαια.

- Δέντρο: Ένα δέντρο είναι ένας συνεκτικός, μη κατευθυνόμενος γράφος χωρίς κύκλους. Ένα δέντρο με n κόμβους περιέχει ακριβώς n-1 ακμές. Μία βασική ιδιότητα των δέντρων είναι ότι μεταξύ οποιωνδήποτε δύο κόμβων υπάρχει μοναδικό μονοπάτι.
- Ριζωμένο δέντρο: Ένα ριζωμένο δέντρο είναι ένα δέντρο στο οποίο ένας κόμβος έχει οριστεί ως ρίζα (root). Η ρίζα αποτελεί την αφετηρία του δέντρου και καθορίζει μία φυσική ιεραρχία μεταξύ των κόμβων. Συγκεκριμένα, για κάθε κόμβο u του δέντρου, το μοναδικό μονοπάτι από τη ρίζα προς τον u ορίζει τους προγόνους του u.

Βασικές Έννοιες:

Έστω ένα ριζωμένο δέντρο T με ρίζα r. Ορίζουμε τις ακόλουθες έννοιες:

- Γονέας: Για κάθε κόμβο u≠r, ο γονέας του u είναι ο αμέσως προηγούμενος κόμβος στο μονοπάτι από τη ρίζα προς τον u.
- Παιδί: Ο κόμβος v είναι παιδί του κόμβου u αν ο u είναι γονέας του v.
- Πρόγονος: Ο κόμβος w είναι πρόγονος του κόμβου v αν βρίσκεται στο μονοπάτι από τη ρίζα r προς τον v. Κάθε κόμβος θεωρείται πρόγονος του εαυτού του.
- Απόγονος: Ο κόμβος v είναι απόγονος του κόμβου w αν ο w είναι πρόγονος του v.
- Βάθος: Το βάθος ενός κόμβου v, συμβολισμένο ως depth(v), είναι ο αριθμός των ακμών στο μοναδικό μονοπάτι από τη ρίζα r προς τον v. Η ρίζα έχει βάθος 0.
- Φύλλο: Ένας κόμβος v ονομάζεται φύλλο αν δεν έχει παιδιά.

- Εσωτερικός κόμβος: Ένας κόμβος v ονομάζεται εσωτερικός αν έχει τουλάχιστον ένα παιδί.
- Υποδέντρο: Το υποδέντρο με ρίζα τον κόμβο v αποτελείται από τον v και όλους τους απόγονούς του.

2.2 Το πρόβλημα Range Minimum Query (RMQ)

Στο πρόβλημα Range Minimum Query (RMQ) δίνεται ένας πίνακας A από n αριθμούς και ζητείται η εύρεση του ελάχιστου στοιχείου σε ένα διάστημα του πίνακα. Ένα ερώτημα RMQ έχει τη μορφή:

$$RMQ(i,j)=\min\{A[i],A[i+1], \dots , A[j]\}$$

για οποιοδήποτε ζεύγος δεικτών $0 \le i \le j \le n-1$.

Σε πολλές εφαρμογές υποβάλλονται πολλαπλά ερωτήματα πάνω στον ίδιο πίνακα. Για αυτόν τον λόγο, ανάλογα με τη μέθοδο που χρησιμοποιείται, μπορεί να προηγείται μία φάση προεπεξεργασίας, κατά την οποία κατασκευάζεται μία βοηθητική δομή δεδομένων που επιτρέπει την αποτελεσματικότερη απάντηση των ερωτημάτων.

Παράδειγμα:

Έστω ο πίνακας A=[3,2,6,1,9]. Μερικά παραδείγματα ερωτημάτων RMQ είναι τα εξής:

RMQ(0,2)= min{3,2,6}= 2

RMQ(1,4)= min{2,6,1,9}= 1

RMQ(0,4)= min{3,2,6,1,9}= 1

Online και Offline RMQ

Μια σημαντική διάκριση στο πρόβλημα RMQ αφορά τον τρόπο με τον οποίο γίνονται διαθέσιμα τα ερωτήματα. Ανάλογα με αυτό, διακρίνουμε την online και offline εκδοχή του προβλήματος.

Στην online εκδοχή τα ερωτήματα γίνονται διαθέσιμα ένα προς ένα και κάθε ερώτημα μπορεί να απαντηθεί χωρίς να είναι απαραίτητη η γνώση των επόμενων ερωτημάτων. Συνήθως προηγείται μία φάση προεπεξεργασίας του πίνακα και στη συνέχεια κάθε ερώτημα απαντάται ανεξάρτητα από τα υπόλοιπα. Η online εκδοχή είναι πιο περιοριστική ως προς τη διαθέσιμη πληροφορία, καθώς δεν είναι δυνατή η εκμετάλλευση πληροφορίας για μελλοντικά ερωτήματα. Ταυτόχρονα είναι πιο

ρεαλιστική για εφαρμογές στις οποίες τα ερωτήματα προκύπτουν δυναμικά κατά τη διάρκεια της εκτέλεσης.

Στην offline εκδοχή το σύνολο των ερωτημάτων είναι γνωστό εκ των προτέρων. Αυτό μπορεί να επιτρέψει την επεξεργασία των ερωτημάτων συνολικά και την αξιοποίηση της μεταξύ τους πληροφορίας, οδηγώντας ενδεχομένως σε αποδοτικότερες λύσεις.

Στην παρούσα εργασία εξετάζονται και οι δύο περιπτώσεις. Η NaiveRMQ, η Sparse Table και οι online προσεγγίσεις που βασίζονται σε Cartesian Tree και LCA επιτρέπουν την ανεξάρτητη απάντηση κάθε ερωτήματος. Παράλληλα, επειδή στα πειράματα τα RMQ queries είναι γνωστά εκ των προτέρων, χρησιμοποιείται και offline προσέγγιση μέσω του Tarjan Offline LCA πάνω στο Cartesian Tree.

Προσεγγίσεις Επίλυσης και Πολυπλοκότητες

Στην παρούσα εργασία υλοποιούμε και συγκρίνουμε διαφορετικές προσεγγίσεις για την επίλυση του προβλήματος RMQ.

Η NaiveRMQ δεν απαιτεί κάποια ιδιαίτερη προεπεξεργασία. Για κάθε ερώτημα RMQ(L,R) εξετάζει διαδοχικά όλα τα στοιχεία του διαστήματος [L,R] και επιστρέφει το ελάχιστο. Ο χρόνος απάντησης είναι ανάλογος με το μήκος του διαστήματος και στη χειρότερη περίπτωση είναι O(n). Η μέθοδος χρησιμοποιείται κυρίως ως λύση αναφοράς για την επαλήθευση της ορθότητας των πιο σύνθετων προσεγγίσεων.

Η SparseTable χρησιμοποιεί προεπεξεργασία κατά την οποία αποθηκεύονται τα ελάχιστα στοιχεία για διαστήματα μήκους δύναμης του 2. Η κατασκευή της απαιτεί χρόνο O(n log n) και χώρο O(n log n). Μετά την προεπεξεργασία, κάθε ερώτημα RMQ μπορεί να απαντηθεί σε χρόνο O(1), χρησιμοποιώντας δύο κατάλληλα επικαλυπτόμενα διαστήματα.

Μία ακόμη προσέγγιση βασίζεται στην αναγωγή του RMQ στο πρόβλημα Lowest Common Ancestor (LCA) μέσω του Cartesian Tree. Το Cartesian Tree κατασκευάζεται σε O(n) χρόνο με χρήση στοίβας. Για κάθε ερώτημα RMQ(i,j), οι θέσεις i και j αντιστοιχούν σε δύο κόμβους του Cartesian Tree και ισχύει $RMQ_A(i,j)=value(LCA_T(node_i,node_j))$.

Στην εργασία διατηρείται μία απλή υλοποίηση LCA, βασισμένη στα βάθη και στους γονείς των κόμβων, η οποία μπορεί στη χειρότερη περίπτωση να απαιτήσει O(n) χρόνο ανά ερώτημα.

Επιπλέον υλοποιείται προσέγγιση βασισμένη στη βέλτιστη static-preprocessed μέθοδο LCA. Η θεωρητική μέθοδος χρησιμοποιεί Euler tour του δέντρου και αναγωγή του LCA σε ειδικής μορφής ±1 RMQ και επιτυγχάνει O(n) προεπεξεργασία και O(1) χρόνο ανά ερώτημα.

Τέλος, για την offline περίπτωση χρησιμοποιείται ο Tarjan Offline LCA. Αφού όλα τα RMQ queries μετατραπούν σε LCA queries πάνω στο Cartesian Tree, επιλύονται ως ενιαίο σύνολο με χρήση Disjoint Set Union. Για n κόμβους και q προκαθορισμένα ερωτήματα, ο συνολικός χρόνος είναι O((n+q)α(n)), όπου α(n) είναι η αντίστροφη συνάρτηση Ackermann.

Προσέγγιση	Προεπεξεργασία/ Συνολική επεξεργασία	Ερώτημα	Χώρος	Τύπος
NaiveRMQ	O(1)	O(n)	O(n)*	Online
Sparse Table	O(n log n)	O(1)	O(n log n)	Online
Cartesian Tree + απλή LCA	O(n)	O(n)	O(n)	Online
Cartesian Tree+ βέλτιστη static-preprocessed LCA	O(n)	O(1)	O(n)	Online
Cartesian Tree + Tarjan Offline LCA	O((n+q)α(n))	-	O(n+q)	Offline

Πίνακας 2.1. Σύγκριση προσεγγίσεων επίλυσης του RMQ

* Στην περίπτωση της NaiveRMQ, ο πίνακας εισόδου απαιτεί O(n) χώρο. Αν υπολογίζουμε μόνο τον επιπλέον βοηθητικό χώρο της μεθόδου, τότε αυτός είναι O(1).

Στον παραπάνω πίνακα, n είναι το πλήθος των στοιχείων του αρχικού πίνακα και q το πλήθος των ερωτημάτων. Ως online χαρακτηρίζονται οι μέθοδοι στις οποίες, μετά την κατάλληλη προεπεξεργασία, κάθε ερώτημα μπορεί να δοθεί και να απαντηθεί ανεξάρτητα από τα υπόλοιπα. Αντίθετα, στην offline προσέγγιση του Tarjan όλα τα ερωτήματα πρέπει να είναι γνωστά εκ των προτέρων και επεξεργάζονται από κοινού κατά τη διάρκεια της διάσχισης του δέντρου.

Στην περίπτωση του Tarjan Offline LCA δεν ορίζεται ανεξάρτητος χρόνος ερωτήματος με τον ίδιο τρόπο όπως στις online μεθόδους, καθώς η επίλυση των ερωτημάτων ενσωματώνεται στη συνολική εκτέλεση του αλγορίθμου. Η επεξεργασία κάθε ερωτήματος περιλαμβάνει πράξεις πάνω στη δομή Union-Find, ενώ ο συνολικός χρόνος για ένα δέντρο με n κόμβους και q προκαθορισμένα ερωτήματα είναι O((n+q)α(n)).

Η α(n) συμβολίζει την αντίστροφη συνάρτηση Ackermann και αυξάνεται εξαιρετικά αργά. Για πρακτικά μεγέθη εισόδου η τιμή της παραμένει πολύ μικρή, γι' αυτό

και οι πράξεις της δομής Union-Find θεωρούνται σχεδόν σταθερού αποσβεσμένου κόστους.

Στην υλοποίηση της Union-Find χρησιμοποιούνται οι τεχνικές path compression και union by rank. Η πρώτη μειώνει το μήκος των διαδρομών προς τη ρίζα κατά τις πράξεις find, ενώ στη δεύτερη κάθε σύνολο διατηρεί μία τιμή rank, η οποία αποτελεί εκτίμηση του ύψους του δέντρου που το αναπαριστά. Κατά την ένωση δύο συνόλων, η ρίζα με το μικρότερο rank συνδέεται κάτω από τη ρίζα με το μεγαλύτερο rank, ώστε τα δέντρα να παραμένουν όσο το δυνατόν πιο ρηχά.

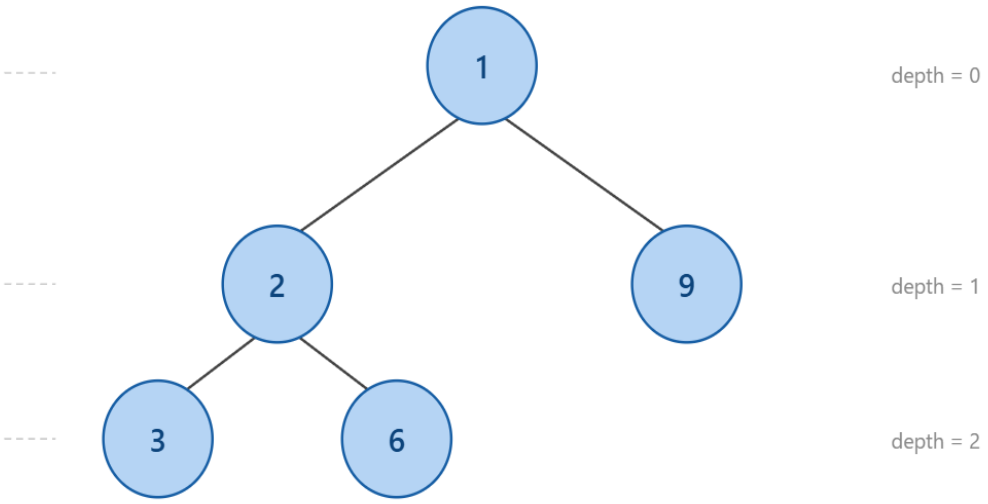
2.3 Το πρόβλημα Lowest Common Ancestor (LCA)

Έστω ένα ριζωμένο δέντρο T με ρίζα r. Για δύο κόμβους u και v του δέντρου, ορίζουμε ως Lowest Common Ancestor (LCA) τον βαθύτερο κόμβο του δέντρου που είναι πρόγονος και των δύο. Συμβολίζουμε τον ελάχιστο κοινό πρόγονο των u και v ως LCA(u,v).

Υπενθυμίζουμε ότι ένας κόμβος w είναι πρόγονος ενός κόμβου u αν το μονοπάτι από τη ρίζα r προς τον u διέρχεται από τον w. Σύμφωνα με αυτόν τον ορισμό, κάθε κόμβος θεωρείται πρόγονος του εαυτού του.

Παράδειγμα:

Έστω το παρακάτω δέντρο με ρίζα τον κόμβο 1 όπως φαίνεται στην εικόνα 2.2:



Εικόνα 2.2. Παράδειγμα δέντρου με ρίζα τον κόμβο 1

Ο πίνακας 2.3 συνοψίζει μερικά παραδείγματα ερωτημάτων LCA:

u	v	LCA(u,v)
3	6	2
3	9	1
6	9	1
2	9	1
3	2	2

Πίνακας 2.3. Παραδείγματα ερωτημάτων LCA για το δέντρο της εικόνας 2.2

Αλγόριθμοι Εύρεσης LCA

Στην εργασία εξετάζονται διαφορετικές προσεγγίσεις για την επίλυση του προβλήματος Lowest Common Ancestor, ανάλογα με το αν τα ερωτήματα δίνονται online ή είναι γνωστά όλα εκ των προτέρων.

Η πρώτη και απλούστερη προσέγγιση βασίζεται στο βάθος κάθε κόμβου και στον δείκτη προς τον γονέα του. Για δύο κόμβους u και v, αρχικά εξισώνονται τα βάθη τους, μετακινώντας τον βαθύτερο κόμβο προς τη ρίζα. Στη συνέχεια οι δύο κόμβοι μετακινούνται ταυτόχρονα προς τους γονείς τους μέχρι να συναντηθούν. Ο κόμβος στον οποίο συναντώνται είναι ο LCA(u,v). Στη συγκεκριμένη υλοποίηση ο χρόνος απάντησης μπορεί στη χειρότερη περίπτωση να είναι O(n).

Για αποδοτικότερη online απάντηση ερωτημάτων χρησιμοποιείται επίσης προσέγγιση βασισμένη στη βέλτιστη static-preprocessed μέθοδο LCA. Αρχικά πραγματοποιείται Euler tour του δέντρου και κατασκευάζεται ο αντίστοιχος πίνακας βαθών. Με τον όρο Euler tour εννοούμε τη διάσχιση του δέντρου κατά την οποία καταγράφεται κάθε κόμβος κάθε φορά που τον επισκεπτόμαστε ή επιστρέφουμε σε αυτόν κατά τη διάσχιση. Για δέντρο με n κόμβους, η ακολουθία που προκύπτει έχει μήκος 2n-1. Κάθε φορά που η διάσχιση μετακινείται από έναν κόμβο σε παιδί ή επιστρέφει στον γονέα, το βάθος μεταβάλλεται κατά ακριβώς +1 ή -1. Επομένως, το πρόβλημα LCA ανάγεται σε ειδικής μορφής ±1 RMQ πάνω στον πίνακα βαθών. Για κάθε κόμβο αποθηκεύεται η πρώτη θέση στην οποία εμφανίζεται κατά το Euler tour. Για δύο κόμβους u και v, εξετάζεται το τμήμα του Euler tour ανάμεσα στις πρώτες εμφανίσεις τους. Ο κόμβος που εμφανίζεται σε θέση με το ελάχιστο βάθος μέσα σε αυτό το τμήμα είναι ο LCA(u,v). Η θεωρητική μέθοδος οργανώνει τον πίνακα σε μικρά blocks και χρησιμοποιεί προϋπολογισμένες απαντήσεις μέσα στα blocks και μία επιπλέον δομή RMQ πάνω στα ελάχιστα των blocks. Με τον τρόπο αυτό επιτυγχάνεται O(n) προεπεξεργασία και O(1) χρόνος ανά ερώτημα LCA.

Για την offline περίπτωση χρησιμοποιείται ο Tarjan Offline LCA. Στην προσέγγιση αυτή όλα τα ερωτήματα είναι γνωστά εκ των προτέρων και επεξεργάζονται κατά τη διάρκεια μίας διάσχισης DFS του δέντρου. Ο αλγόριθμος χρησιμοποιεί δομή Disjoint Set Union (Union-Find) για τη διαχείριση των συνόλων κόμβων που έχουν ήδη επεξεργαστεί. Κατά τη DFS, αφού ολοκληρωθεί η επεξεργασία ενός υποδέντρου, το σύνολο των κόμβων του συγχωνεύεται με το σύνολο του γονέα του. Με αυτόν τον τρόπο, η Union-Find επιτρέπει να εντοπίζεται αποδοτικά ο κατάλληλος αντιπρόσωπος του συνόλου και, μέσω αυτού, ο κοινός πρόγονος που απαιτείται για τα offline ερωτήματα. Με χρήση path compression και union by rank, οι οποίες παρουσιάστηκαν στην προηγούμενη ενότητα, για δέντρο με n κόμβους και q προκαθορισμένα ερωτήματα ο συνολικός χρόνος είναι $O((n+q)\alpha(n))$.

Σχέση LCA και RMQ

Η σχέση μεταξύ LCA και RMQ είναι θεμελιώδης, καθώς τα δύο προβλήματα μπορούν να αναχθούν το ένα στο άλλο.

Συγκεκριμένα, για έναν πίνακα A κατασκευάζουμε ένα δέντρο (το Cartesian Tree του A) με την εξής ιδιότητα: Για κάθε θέση i του πίνακα υπάρχει ένας αντίστοιχος κόμβος $node_i$ στο Cartesian Tree. Για οποιοδήποτε ζεύγος δεικτών $i \leq j$, ισχύει:

$$RMQ_A(i,j)=value(LCA_T(node_i,node_j))$$

όπου $RMQ_A(i,j)$ είναι το ελάχιστο στοιχείο του πίνακα A στο διάστημα [i, j] και $LCA_T(node_i,node_j)$ είναι ο ελάχιστος κοινός πρόγονος των κόμβων που αντιστοιχούν στις θέσεις i και j στο Cartesian Tree T.

Η αντιστοίχιση γίνεται με βάση τις θέσεις των στοιχείων και όχι αποκλειστικά στις τιμές τους, καθώς διαφορετικές θέσεις του πίνακα μπορεί να περιέχουν ίσες τιμές.

Η αντίστροφη αναγωγή, από LCA σε RMQ, πραγματοποιείται μέσω Euler tour ενός ριζωμένου δέντρου. Από το Euler tour κατασκευάζεται ένας πίνακας με τα βάθη των κόμβων, στον οποίο ο LCA δύο κόμβων αντιστοιχεί στο στοιχείο ελάχιστου βάθους μεταξύ των πρώτων εμφανίσεών τους. Επειδή οι διαδοχικές τιμές του πίνακα βαθών διαφέρουν κατά +1 ή -1, το RMQ που προκύπτει έχει ειδική μορφή ±1 RMQ.

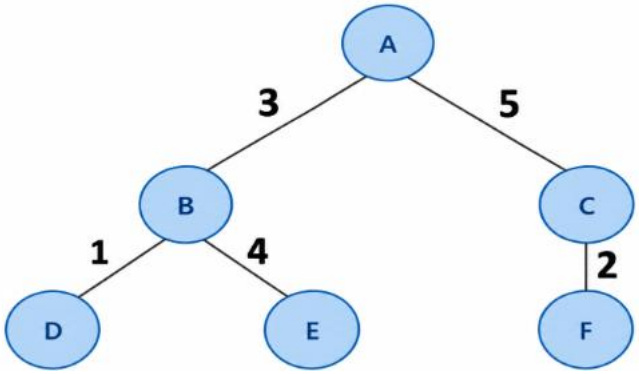
2.4 Το πρόβλημα Bottleneck Edge Query (BEQ)

Έστω ένα μη κατευθυνόμενο δέντρο T με βάρη στις ακμές. Για δύο κόμβους u και v του δέντρου, ορίζουμε ως Bottleneck Edge Query (BEQ) το ερώτημα εύρεσης της ακμής με το ελάχιστο βάρος στο μοναδικό μονοπάτι μεταξύ των u και v. Συμβολίζουμε το αποτέλεσμα ως BEQ(u,v).

Η έννοια αυτή μπορεί να ερμηνευτεί μέσω προβλημάτων χωρητικότητας δικτύων. Αν κάθε ακμή αναπαριστά έναν αγωγό με χωρητικότητα ίση με το βάρος της, τότε η μέγιστη ποσότητα που μπορεί να μεταφερθεί κατά μήκος του μοναδικού μονοπατιού μεταξύ u και v περιορίζεται από την ακμή με τη μικρότερη χωρητικότητα, δηλαδή από την ακμή με το ελάχιστο βάρος στο μονοπάτι.

Παράδειγμα:

Έστω το δέντρο της εικόνας 2.4 με βάρη στις ακμές:



Εικόνα 2.4. Παράδειγμα δέντρου με βάρη στις ακμές

Ο πίνακας 2.5 συνοψίζει μερικά παραδείγματα ερωτημάτων BEQ.

u	v	Μονοπάτι	BEQ(u,v)
D	F	D→B→A→C→F	ακμή B-D (βάρος=1)
D	E	D→B→E	ακμή B-D (βάρος=1)
E	F	E→B→A→C→F	ακμή C-F (βάρος=2)
A	D	A→B→D	ακμή B-D (βάρος=1)
A	F	A→C→F	ακμή C-F (βάρος=2)
B	C	B→A→C	ακμή A-B (βάρος=3)

Πίνακας 2.5. Παραδείγματα ερωτημάτων BEQ για το δέντρο της εικόνας 2.4

Σχέση BEQ και RMQ

Το BEQ αποτελεί φυσική γενίκευση του RMQ από πίνακες σε δέντρα. Ακριβώς όπως το RMQ βρίσκει το ελάχιστο στοιχείο σε ένα διάστημα πίνακα, το BEQ βρίσκει την ακμή με το ελάχιστο βάρος στο μονοπάτι μεταξύ δύο κόμβων ενός δέντρου.

Η σχέση αυτή προκύπτει από τη δυνατότητα αναγωγής και των δύο προβλημάτων σε ερωτήματα LCA μέσω των αντίστοιχων Cartesian δομών. Όπως ένα ερώτημα RMQ μπορεί να αναχθεί σε ερώτημα LCA μέσω του Cartesian Tree, έτσι και ένα ερώτημα BEQ μπορεί να αναχθεί σε ερώτημα LCA μέσω του Cartesian Tree of Tree.

Συγκεκριμένα, για κάθε δέντρο T με βάρη στις ακμές, κατασκευάζουμε ένα Cartesian Tree of Tree C με την εξής ιδιότητα:

$$BEQ_T(u,v)=edge(LCA_C(leaf_u,leaf_v))$$

όπου $leaf_u$ και $leaf_v$ είναι τα φύλλα του C που αντιστοιχούν στους κόμβους u και v του αρχικού δέντρου T, και $edge(LCA_C(leaf_u,leaf_v))$ είναι η ακμή του αρχικού δέντρου που αντιστοιχεί στον εσωτερικό κόμβο $LCA_C(leaf_u,leaf_v)$. Στην παρούσα εργασία εξετάζεται τόσο η online απάντηση των BEQ ερωτημάτων μέσω LCA πάνω στο Cartesian Tree of Tree, όσο και η offline περίπτωση, όταν όλα τα ερωτήματα είναι γνωστά εκ των προτέρων. Στη δεύτερη περίπτωση, τα BEQ ερωτήματα μετατρέπονται σε LCA ερωτήματα πάνω στο CToT και επεξεργάζονται από κοινού με τον Tarjan Offline LCA. Η κατασκευή και η λειτουργία του Cartesian Tree of Tree περιγράφονται αναλυτικά στο κεφάλαιο 5.

Κεφάλαιο 3. Cartesian Tree

3.1 Ορισμός και Ιδιότητες

Το Cartesian Tree είναι μία δομή δεδομένων που κατασκευάζεται από έναν πίνακα A με n στοιχεία και συνδυάζει δύο βασικές ιδιότητες: την ιδιότητα σωρού (heap property) και την ιδιότητα διάταξης των θέσεων του πίνακα, η οποία αντιστοιχεί στην inorder διάσχιση του δέντρου.

Ορισμός: Έστω πίνακας A[0..n-1]. Το Cartesian Tree του A είναι ένα δυαδικό δέντρο T που ορίζεται αναδρομικά ως εξής:

- Η ρίζα του T είναι ο κόμβος που αντιστοιχεί στο ελάχιστο στοιχείο A[i], δηλαδή A[i]=min{A[0], … , A[n-1]}.
- Το αριστερό υποδέντρο είναι το Cartesian Tree του υποπίνακα A[0..i -1].
- Το δεξί υποδέντρο είναι το Cartesian Tree του υποπίνακα A[i+1...n-1].

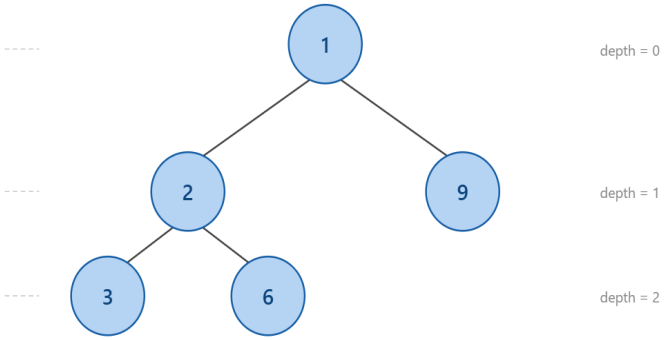
Ιδιότητες: Το Cartesian Tree ικανοποιεί ταυτόχρονα δύο ιδιότητες:

- Ιδιότητα σωρού: Κάθε κόμβος του δέντρου έχει τιμή μικρότερη ή ίση από τις τιμές των παιδιών του. Συνεπώς, η ρίζα του δέντρου αντιστοιχεί πάντα στο ελάχιστο στοιχείο του πίνακα.
- Ιδιότητα δυαδικού δέντρου αναζήτησης: Η διάταξη των κόμβων στο δέντρο αντικατοπτρίζει τη σειρά των αντίστοιχων στοιχείων στον πίνακα. Συγκεκριμένα για κάθε κόμβο v με αντίστοιχη θέση i στον πίνακα, όλοι οι κόμβοι του αριστερού υποδέντρου αντιστοιχούν σε θέσεις μικρότερες του i, ενώ όλοι οι κόμβοι του δεξιού υποδέντρου αντιστοιχούν σε θέσεις μεγαλύτερες του i.

Μία άμεση συνέπεια της παραπάνω ιδιότητας διάταξης είναι ότι η inorder διάσχιση του Cartesian Tree επιστρέφει τα στοιχεία με την ίδια σειρά με την οποία εμφανίζονται στον αρχικό πίνακα A.

Παράδειγμα:

Έστω ο πίνακας A=[3,2,6,1,9]. Το Cartesian Tree του πίνακα A φαίνεται στην εικόνα 3.1.



Εικόνα 3.1. Cartesian Tree του πίνακα A=[3,2,6,1,9]

Παρατηρούμε ότι:

- ✓ Η ρίζα είναι ο κόμβος με τιμή 1 (ελάχιστο στοιχείο, θέση 3).
- ✓ Στο αριστερό υποδέντρο της ρίζας βρίσκονται οι κόμβοι με τιμές 3,2,6 (θέσεις 0,1,2).
- ✓ Στο δεξί υποδέντρο της ρίζας βρίσκεται ο κόμβος με τιμή 9 (θέση 4).
- ✓ Η inorder διάσχιση, δηλαδή αριστερό υποδέντρο – κόμβος – δεξί υποδέντρο, επιστρέφει τη σειρά 3, 2, 6, 1, 9, δηλαδή τη σειρά των στοιχείων του αρχικού πίνακα.

3.2 Κατασκευή

Ο αλγόριθμος κατασκευής του Cartesian Tree από έναν πίνακα A[0..n-1] βασίζεται στη χρήση μίας στοίβας. Τα στοιχεία του πίνακα εξετάζονται διαδοχικά από αριστερά προς τα δεξιά και για κάθε στοιχείο δημιουργείται ο αντίστοιχος κόμβος του δέντρου.

Η στοίβα χρησιμοποιείται ώστε να διατηρούνται οι κόμβοι που έχουν ήδη εξεταστεί και μπορούν να αποτελέσουν προγόνους των επόμενων στοιχείων. Για κάθε νέο στοιχείο αφαιρούνται από την κορυφή της στοίβας όλοι οι κόμβοι με μεγαλύτερη τιμή. Ο τελευταίος κόμβος που αφαιρέθηκε γίνεται το αριστερό παιδί του νέου κόμβου. Αν μετά τις αφαιρέσεις η στοίβα δεν είναι κενή, ο νέος κόμβος γίνεται το δεξί παιδί του κόμβου που βρίσκεται στην κορυφή της. Στη συνέχεια ο νέος κόμβος εισάγεται στη στοίβα.

Με τον τρόπο αυτό διατηρούνται τόσο η ιδιότητα σωρού όσο και η σειρά των στοιχείων του αρχικού πίνακα κατά την inorder διάσχιση.

Ψευδοκώδικας

```
Algorithm BuildCartesianTree(A,n)
stack ← empty stack
for i ← 0 to n - 1 do
        current ← new Node(A[i], i)
        last ← null
        while stack is not empty and value(top(stack)) > A[i] do
                last ← pop(stack)
        current.left ← last
        if stack is not empty then
                top(stack).right ← current
        push(stack, current)
root ← bottom(stack)
```

Ο παραπάνω ψευδοκώδικας αντιστοιχεί στη λογική της υλοποίησης που χρησιμοποιείται στην εργασία. Η αυστηρή σύγκριση > διατηρεί τη συμπεριφορά της υλοποίησης και στην περίπτωση εμφάνισης ίσων τιμών.

Παράδειγμα βήμα προς βήμα

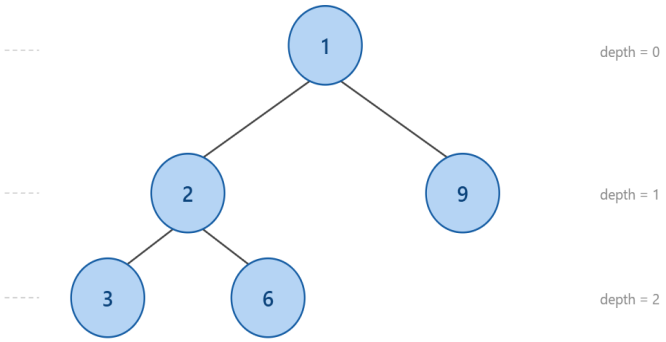
Έστω ο πίνακας A= [3,2,6,1,9]. Η κατασκευή εκτελείται ως εξής:

1) Επεξεργασία του A[0]= 3: Δημιουργείται ο κόμβος 3. Η στοίβα είναι κενή, επομένως δεν αφαιρείται κανένας κόμβος. Ο 3 εισάγεται στη στοίβα (stack=[3]). Το δέντρο αποτελείται αρχικά μόνο από τον κόμβο 3.
2) Επεξεργασία του A[1]= 2: Η κορυφή της στοίβας είναι ο κόμβος 3. Επειδή 3>2, ο 3 αφαιρείται από τη στοίβα. Είναι ο τελευταίος κόμβος που αφαιρέθηκε και επομένως γίνεται το αριστερό παιδί του 2. Η στοίβα έχει πλέον αδειάσει και ο κόμβος 2 εισάγεται σε αυτήν (stack=[2]).
3) Επεξεργασία του A[2]=6: Η κορυφή της στοίβας είναι ο κόμβος 2. Επειδή 2 <6 δεν αφαιρείται κανένας κόμβος. Ο 6 γίνεται το δεξί παιδί του 2 και εισάγεται στη στοίβα (stack=[2,6]).
4) Επεξεργασία του A[3]=1: Αρχικά η κορυφή της στοίβας είναι ο 6 και ισχύει 6>1, οπότε ο 6 αφαιρείται. Στη συνέχεια η κορυφή είναι ο 2 και ισχύει 2>1, οπότε αφαιρείται και ο 2. Ο τελευταίος κόμβος που αφαιρέθηκε είναι ο 2. Επομένως ο 2 γίνεται το αριστερό παιδί του νέου κόμβου 1. Το υποδέντρο που έχει

δημιουργηθεί κάτω από τον κόμβο 2 διατηρείται, δηλαδή ο 2 εξακολουθεί να έχει αριστερό παιδί τον 3 και δεξί παιδί τον 6. Η στοίβα έχει πλέον αδειάσει και ο κόμβος 1 εισέρχεται σε αυτήν (stack=[1]).

5) Επεξεργασία του A[4]=9: Η κορυφή της στοίβας είναι ο κόμβος 1. Επειδή 1<9, δεν αφαιρείται κανένας κόμβος. Ο 9 γίνεται δεξί παιδί του 1 και εισάγεται στη στοίβα (stack=[1,9]).

Το τελικό Cartesian Tree που προκύπτει από την κατασκευή φαίνεται στην εικόνα 3.2.



Εικόνα 3.2. Αποτέλεσμα παραδείγματος κατασκευής Cartesian Tree

Πολυπλοκότητα

	Πολυπλοκότητα
Χρόνος κατασκευής	O(n)
Χώρος	O(n)

Πίνακας 3.3. Πολυπλοκότητα αλγορίθμου κατασκευής Cartesian Tree με χρήση στοίβας

Κάθε στοιχείο του πίνακα δημιουργεί έναν κόμβο, ο οποίος εισάγεται στη στοίβα ακριβώς μία φορά. Παράλληλα, κάθε κόμβος μπορεί να αφαιρεθεί από τη στοίβα το πολύ μία φορά. Επομένως, παρότι στον αλγόριθμο υπάρχει βρόχος while μέσα στον βρόχο for, ο συνολικός αριθμός πράξεων εισαγωγής και αφαίρεσης από τη στοίβα είναι O(n).

Κατά συνέπεια, ο συνολικός χρόνος κατασκευής του Cartesian Tree είναι O(n). Ο βοηθητικός χώρος που απαιτείται για τη στοίβα μπορεί στη χειρότερη περίπτωση να περιέχει n κόμβους και επομένως είναι O(n).

3.3 Σύνδεση RMQ και LCA

Όπως αναφέρθηκε και στην Ενότητα 2.3, υπάρχει στενή σχέση μεταξύ των προβλημάτων RMQ και LCA. Στην παρούσα ενότητα παρουσιάζουμε αναλυτικά τον τρόπο με τον οποίο το Cartesian Tree επιτρέπει την αναγωγή ενός ερωτήματος RMQ σε ερώτημα LCA.

Η Βασική Ιδιότητα

Έστω ένας πίνακας A[0.. n-1] και το Cartesian Tree T του A. Για οποιοδήποτε ζεύγος δεικτών i≤j, ισχύει:

$$RMQ_A(i,j)=value(LCA_T(node_i,node_j))$$

όπου $node_i$ και $node_j$ είναι οι κόμβοι του T που αντιστοιχούν στις θέσεις i και j του πίνακα A, και value(v) είναι η τιμή που αποθηκεύεται στον κόμβο v.

Απόδειξη:

Έστω k η θέση του ελάχιστου στοιχείου στο διάστημα [i,j], δηλαδή A[k]= $RMQ_A(i,j)$.

Βήμα 1: Δείχνουμε ότι ο $node_k$ είναι κοινός πρόγονος των $node_i$ και $node_j$.

Από την ιδιότητα inorder του Cartesian Tree, οι κόμβοι που αντιστοιχούν στις θέσεις ενός συνεχόμενου διαστήματος του πίνακα διατηρούν την ίδια σχετική διάταξη στο δέντρο. Ο κόμβος $node_k$, που αντιστοιχεί στο ελάχιστο στοιχείο του διαστήματος [i,j], αποτελεί τη ρίζα του τμήματος του Cartesian Tree που περιέχει τους κόμβους των θέσεων i, ... ,j. Επομένως, είναι κοινός πρόγονος των $node_i$ και $node_j$.

Βήμα 2: Δείχνουμε ότι ο $node_k$ είναι ο βαθύτερος κοινός πρόγονος.

Έστω ότι υπάρχει βαθύτερος κοινός πρόγονος $node_m$ των $node_i$ και $node_j$. Λόγω της inorder διάταξης, ο δείκτης m θα πρέπει να ανήκει στο διάστημα [i,j]. Επειδή όμως ο $node_k$ αντιστοιχεί στο ελάχιστο στοιχείο του διαστήματος και το Cartesian Tree ικανοποιεί την ιδιότητα σωρού, ο $node_k$ βρίσκεται πάνω από κάθε κόμβο του ίδιου διαστήματος με μεγαλύτερη τιμή. Συνεπώς δεν μπορεί να υπάρχει βαθύτερος κοινός πρόγονος των $node_i$ και $node_j$, και επομένως:

$$LCA_T(node_i,node_j)=node_k$$

Βήμα 3: Συμπέρασμα

Από τα βήματα 1 και 2 προκύπτει:

$$value(LCA_T(node_i,node_j))=value(node_k)=A[k]=RMQ_A(i,j)$$

Παράδειγμα:

Έστω ο πίνακας A=[3,2,6,1,9] και το Cartesian Tree της εικόνας 3.2. Εξετάζουμε τα ακόλουθα ερωτήματα:

Ερώτημα 1: RMQ(0,2)= min{3,2,6}=2

LCA($node_0$,$node_2$)= κόμβος με τιμή 2

Ερώτημα 2: RMQ(0,4)= min{3,2,6,1,9}=1

LCA($node_0$,$node_4$)= κόμβος με τιμή 1

Ερώτημα 3: RMQ(1,3)= min{2,6,1}=1

LCA($node_1$,$node_3$)= κόμβος με τιμή 1

Παρατηρούμε ότι σε κάθε περίπτωση το αποτέλεσμα του $RMQ_A(i,j)$ συμπίπτει με την τιμή του $LCA_T(node_i,node_j)$ στο Cartesian Tree, επιβεβαιώνοντας στο συγκεκριμένο παράδειγμα τη σχέση μεταξύ RMQ και LCA μέσω του Cartesian Tree.

Αλγόριθμος απάντησης ερωτήματος:

Για την απάντηση ενός ερωτήματος RMQ(i,j) μέσω του Cartesian Tree ακολουθούμε τα εξής βήματα:

Βήμα 1: Από τις θέσεις i και j του αρχικού πίνακα εντοπίζουμε τους κόμβους $node_i$ και $node_j$ στο Cartesian Tree.

Βήμα 2: Υπολογίζουμε τον $LCA_T(node_i,node_j)$. Ο τρόπος υπολογισμού του εξαρτάται από τη μέθοδο LCA που χρησιμοποιείται. Στην απλή online μέθοδο οι κόμβοι μετακινούνται προς τους γονείς μέχρι να συναντηθούν, ενώ στην static-preprocessed μέθοδο το ερώτημα απαντάται μέσω της προϋπολογισμένης δομής που βασίζεται στο Euler tour. Στην offline περίπτωση, όλα τα αντίστοιχα LCA ερωτήματα είναι γνωστά εκ των προτέρων και επιλύονται από κοινού με τον Tarjan Offline LCA.

Βήμα 3: Επιστρέφουμε την τιμή που αποθηκεύεται στον κόμβο $LCA_T(node_i,node_j)$. Η ορθότητα του τελευταίου βήματος προκύπτει από τη βασική ιδιότητα του Cartesian Tree που αποδείχθηκε προηγουμένως.

Algorithm RMQUsingCartesianTree(i,j)

u ← node corresponding to position i

v ← node corresponding to position j

w ← LCA(u,v)

return value(w)

Κεφάλαιο 4. Sparse Table

Στο κεφάλαιο αυτό παρουσιάζουμε το Sparse Table, μια δομή δεδομένων που επιτρέπει την απάντηση RMQ ερωτημάτων σε χρόνο O(1) μετά από προεπεξεργασία O(n log n). Αρχικά παρουσιάζουμε τη βασική ιδέα και τον ορισμό της δομής, στη συνέχεια αναλύουμε την κατασκευή της και τέλος παρουσιάζουμε τον τρόπο απάντησης ερωτημάτων.

4.1 Εισαγωγή και Βασική Ιδέα

Το Sparse Table είναι μία δομή δεδομένων που χρησιμοποιείται για την αποδοτική απάντηση ερωτημάτων σε στατικούς πίνακες, δηλαδή σε πίνακες των οποίων οι τιμές δεν μεταβάλλονται μετά την προεπεξεργασία. Στην παρούσα εργασία χρησιμοποιούμε το Sparse Table για την απάντηση ερωτημάτων Range Minimum Query (RMQ).

Η βασική ιδέα της δομής είναι να προϋπολογίσουμε την ελάχιστη τιμή για όλα τα διαστήματα των οποίων το μήκος είναι δύναμη του 2. Δηλαδή, αποθηκεύουμε αποτελέσματα για διαστήματα μήκους 1,2,4,8, ... ,2^k.

Με αυτόν τον τρόπο, όταν θέλουμε να απαντήσουμε ένα ερώτημα RMQ(L,R), δεν χρειάζεται να εξετάσουμε διαδοχικά όλα τα στοιχεία του διαστήματος [L,R]. Αντίθετα, χρησιμοποιούμε δύο προϋπολογισμένες τιμές που αντιστοιχούν σε κατάλληλα διαστήματα.

Έστω $k=\lfloor \log_2(R-L+1) \rfloor$

Τότε επιλέγουμε δύο διαστήματα μήκους 2^k: το πρώτο ξεκινά από τη θέση L, ενώ το δεύτερο καταλήγει στη θέση R. Τα δύο διαστήματα μπορεί να επικαλύπτονται, όμως αυτό δεν επηρεάζει το αποτέλεσμα στην πράξη του ελαχίστου. Αυτό συμβαίνει επειδή η πράξη του ελαχίστου είναι ισοδύναμη, δηλαδή min(x,x)=x.

Επομένως, ακόμη και αν κάποιο στοιχείο του αρχικού διαστήματος περιλαμβάνεται και στα δύο επιλεγμένα υποδιαστήματα, η επανάληψή του δεν μεταβάλλει την τελική ελάχιστη τιμή.

Έτσι, ένα ερώτημα RMQ(L,R) μπορεί να απαντηθεί χρησιμοποιώντας μόνο δύο προϋπολογισμένες τιμές, γεγονός που επιτρέπει χρόνο απάντησης O(1) μετά την κατασκευή της δομής.

4.2 Κατασκευή

Ορισμός:

Ορίζουμε τον δισδιάστατο πίνακα st[i][j] ως εξής:

$st[i][j]=\min\{A[j],A[j+1], \dots , A[j+2^i-1]\}$

δηλαδή το st[i][j] αποθηκεύει το ελάχιστο του διαστήματος μήκους 2^i που ξεκινάει από τη θέση j.

Αναδρομικός Τύπος

Ο πίνακας st υπολογίζεται αναδρομικά ως εξής:

Βάση: Για i=0 (διάστημα μήκους $2^0=1$):

$$st[0][j]= A[j], \quad \forall j \in [0, n-1]$$

Αναδρομή: Για i>0, κάθε διάστημα μήκους 2^i χωρίζεται σε δύο διαδοχικά υποδιαστήματα μήκους 2^{i-1}:

$$st[i][j]= \min(st[i-1][j], st[i-1][j+ 2^{i-1}])$$

Ψευδοκώδικας

Algorithm BuildSparseTable(A,n)

for j←0 to n-1 do

st[0][j] ← A[j]

for i← 1 to $\lfloor \log_2 n \rfloor$ do

for j←0 to n- 2^i do

$st[i][j] \leftarrow \min(st[i-1][j], st[i-1][j+ 2^{i-1}])$

Παράδειγμα βήμα προς βήμα

Έστω ο πίνακας A= [3,2,6,1,9], n=5.

Επίπεδο 0 (i=0, μήκος $2^0=1$):

st[0]=[3,2,6,1,9]

Επίπεδο 1 (i=1, μήκος $2^1=2$):

st[1][0]= min(st[0][0], st[0][1])= min(3,2)= 2

st[1][1]= min(st[0][1], st[0][2])= min(2,6)= 2

st[1][2]= min(st[0][2], st[0][3])= min(6,1)= 1

st[1][3]= min(st[0][3], st[0][4])= min(1,9)= 1

- st[1]=[2,2,1,1]

Επίπεδο 2 (i=2, μήκος 2^2= 4):

st[2][0]= min(st[1][0], st[1][2])= min(2,1)= 1

st[2][1]= min(st[1][1], st[1][3])= min(2,1) =1

- st[2]=[1,1]

Ο πίνακας 4.1 συνοψίζει τον πίνακα st για τον πίνακα A=[3,2,6,1,9]:

Επίπεδο i	Μήκος 2^i	st[i][0]	st[i][1]	st[i][2]	st[i][3]	st[i][4]
0	1	3	2	6	1	9
1	2	2	2	1	1	-
2	4	1	1	-	-	-

Πίνακας 4.1. Πίνακας st για τον πίνακα A=[3,2,6,1,9]

Πολυπλοκότητα

	Τιμή
Χρόνος κατασκευής	O(n log n)
Χώρος	O(n log n)

Πίνακας 4.2. Πολυπλοκότητα κατασκευής Sparse Table

Ο χρόνος κατασκευής είναι O(n log n), καθώς υπάρχουν $\lfloor \log_2 n \rfloor + 1$ επίπεδα και σε κάθε επίπεδο υπολογίζονται το πολύ n έγκυρες τιμές. Επομένως, ο συνολικός αριθμός υπολογισμών είναι O(n log n). Το συνολικό πλήθος υπολογισμών είναι:

$$\sum_{i=1}^{\lfloor \log 2n \rfloor} (n - 2^i + 1) = O(n \log n)$$

Ο χώρος είναι επίσης O(n log n), καθώς η υλοποίηση αποθηκεύει έναν δισδιάστατο πίνακα με $\lfloor \log_2 n \rfloor + 1$ γραμμές και n στήλες. Για παράδειγμα, για n=5 υπάρχουν 3 επίπεδα και συνολικά 5+4+2= 11 έγκυρες τιμές.

Αξίζει να σημειωθεί ότι παρόλο που ο χώρος O(n log n) είναι μεγαλύτερος από τον γραμμικό χώρο O(n) άλλων δομών, στην πράξη παραμένει αποδεκτός για τυπικά μεγέθη πινάκων. Ωστόσο, το βασικό πλεονέκτημα του Sparse Table είναι ότι, μετά την προεπεξεργασία, κάθε ερώτημα RMQ απαντάται σε χρόνο O(1), όπως αναλύεται στην επόμενη ενότητα.

4.3 Απάντηση Ερωτημάτων

Η Ιδέα των Επικαλυπτόμενων Διαστημάτων

Για την απάντηση ενός ερωτήματος RMQ(L,R) εκμεταλλευόμαστε την ιδιότητα που αναφέρθηκε στην ενότητα 4.1, σύμφωνα με την οποία το ελάχιστο δεν επηρεάζεται από την επανάληψη στοιχείων. Αυτό μας επιτρέπει να χρησιμοποιήσουμε δύο προϋπολογισμένες τιμές που αντιστοιχούν σε διαστήματα ίδιου μήκους, τα οποία μπορεί να επικαλύπτονται. Θέτουμε $k=\lfloor \log_2(R-L+1) \rfloor$. Τότε χρησιμοποιούμε δύο διαστήματα μήκους 2^k.

Συγκεκριμένα:

- Το αριστερό διάστημα ξεκινάει από τη θέση L και έχει μήκος 2^k:$[L, L+2^k-1]$.
- Το δεξί διάστημα τελειώνει στη θέση R και έχει μήκος 2^k :$[R- 2^k+1, R]$.

Η ένωση των δύο διαστημάτων καλύπτει ολόκληρο το [L,R]. Επειδή η πράξη του ελαχίστου δεν επηρεάζεται από την πιθανή επικάλυψη, η απάντηση δίνεται από:

$$RMQ(L,R)= \min(st[k][L], st[k][R- 2^k+1])$$

Ψευδοκώδικας

Algorithm Query (L,R)

$k \leftarrow \lfloor \log_2(R-L+1) \rfloor$

return $\min(st[k][L], st[k][R-2^k+1])$

Παράδειγμα

Έστω ο πίνακας A=[3,2,6,1,9] και ο πίνακας st του Πίνακα 4.1. Εξετάζουμε τα ακόλουθα ερωτήματα:

Ερώτημα 1: RMQ(0,2)

Μήκος= 2-0+1=3, $k=\lfloor \log_2 3 \rfloor=1$

$RMQ(0,2)=\min(st[1][0], st[1][2-2+1]) =\min(st[1][0],st[1][1])=\min(2,2)=2$

Ερώτημα 2: RMQ(1,4)

Μήκος= 4-1+1=4, $k=\lfloor \log_2 4 \rfloor=2$

$RMQ(1,4)=\min(st[2][1], st[2][4-4+1]) =\min(st[2][1],st[2][1])=\min(1,1)=1$

Ερώτημα 3: RMQ(0,3)

Μήκος= 3-0+1=4, $k=\lfloor \log_2 4 \rfloor=2$

$RMQ(0,3)=\min(st[2][0], st[2][3-4+1]) =\min(st[2][0],st[2][0])=\min(1,1)=1$

Παρατηρούμε ότι στα συγκεκριμένα παραδείγματα τα αποτελέσματα συμφωνούν με τα αναμενόμενα, παρέχοντας μία άμεση επαλήθευση της λειτουργίας της μεθόδου.

Πολυπλοκότητα

Ένα από τα σημαντικότερα πλεονεκτήματα του Sparse Table είναι ότι κάθε ερώτημα RMQ(L,R) απαντάται σε σταθερό χρόνο O(1), ανεξάρτητα από το μέγεθος του διαστήματος [L,R] και το πλήθος των στοιχείων του πίνακα.

Αυτό επιτυγχάνεται επειδή η απάντηση κάθε ερωτήματος απαιτεί έναν σταθερό αριθμό πράξεων:

Βήμα 1: Υπολογισμός του $k=\lfloor \log_2(R-L+1) \rfloor$ σε χρόνο O(1) με χρήση του προϋπολογισμένου πίνακα λογαρίθμων, ο οποίος παρουσιάζεται στην ενότητα 4.4.

Βήμα 2: Προσπελαύνουμε τις δύο προϋπολογισμένες τιμές, st[k][L] και $st[k][R-2^k+1]$, κάθε μία σε χρόνο O(1).

Βήμα 3: Συγκρίνουμε τις δύο τιμές και επιστρέφουμε τη μικρότερη, επίσης σε χρόνο O(1).

Συνεπώς, ο συνολικός χρόνος απάντησης ενός ερωτήματος είναι O(1). Αυτό αποτελεί σημαντική βελτίωση σε σχέση με τη NaiveRMQ, η οποία απαιτεί στη χειρότερη περίπτωση O(n) χρόνο ανά ερώτημα. Το κόστος αυτής της βελτίωσης είναι η προεπεξεργασία O(n log n) που απαιτείται για την κατασκευή του πίνακα st, η οποία εκτελείται μία φορά πριν από την απάντηση των ερωτημάτων.

4.4 Προϋπολογισμός Λογαρίθμων

Για την απάντηση κάθε ερωτήματος RMQ(L,R) χρειαζόμαστε την τιμή $k= \lfloor \log_2(R-L+1) \rfloor$. Επειδή η τιμή αυτή χρησιμοποιείται σε κάθε ερώτημα, στην υλοποίησή μας την προϋπολογίζουμε για όλα τα πιθανά μήκη διαστημάτων. Συγκεκριμένα, υπολογίζουμε τις τιμές $\lfloor \log_2(x) \rfloor$ για κάθε $x \in [1,n]$ και τις αποθηκεύουμε σε έναν πίνακα $\log_2[x]$. Με αυτόν τον τρόπο, κατά την απάντηση ενός ερωτήματος, η κατάλληλη τιμή του k ανακτάται άμεσα από τον πίνακα χωρίς να απαιτείται νέος υπολογισμός του λογαρίθμου.

Αναδρομικός Τύπος

Ο πίνακας $\log_2$ υπολογίζεται αναδρομικά εκμεταλλευόμενος την εξής ιδιότητα:

$$\lfloor \log_2(x) \rfloor = \lfloor \log_2(\lfloor x/2 \rfloor) \rfloor + 1, \; x \geq 2.$$

όπου η σχέση αυτή προκύπτει από το γεγονός ότι η ακέραια διαίρεση του x με το 2 μειώνει κατά μία τη μεγαλύτερη δύναμη του 2 που δεν υπερβαίνει το x. Η βάση της αναδρομής είναι:

$$\log_2[1]=0$$

Παράδειγμα

Για n=9 ο πίνακας $\log_2$ υπολογίζεται ως εξής:

x	⌊x/2⌋	$\log_2[x]$	$\lfloor\log_2(x)\rfloor$
1	-	0	0
2	1	1	1
3	1	1	1
4	2	2	2
5	2	2	2
6	3	2	2
7	3	2	2
8	4	3	3
9	4	3	3

Πίνακας 4.3. Προϋπολογισμένες τιμές $\log_2[x]$ για $x \in [1,9]$

Παρατηρούμε ότι οι τιμές του πίνακα $\log_2$ συμπίπτουν με τις τιμές $\lfloor\log_2(x)\rfloor$ για κάθε $x \in [1,9]$, επιβεβαιώνοντας τον τρόπο υπολογισμού του πίνακα. Επιπλέον παρατηρούμε ότι η τιμή $\log_2[x]$ αυξάνεται κατά 1 κάθε φορά που το x γίνεται δύναμη του 2, γεγονός που αντικατοπτρίζει τη λογαριθμική φύση της συνάρτησης.

Πολυπλοκότητα

Ο προϋπολογισμός του πίνακα log2 απαιτεί χρόνο O(n) και χώρο O(n), καθώς υπολογίζεται μία τιμή για κάθε $x \in [1,n]$, ενώ κάθε νέα τιμή προκύπτει σε χρόνο O(1) από μία προηγουμένως υπολογισμένη τιμή. Η προσθήκη αυτού του προϋπολογισμού δεν μεταβάλλει τη συνολική ασυμπτωτική πολυπλοκότητα της προεπεξεργασίας του Sparse Table. Η κατασκευή του πίνακα st απαιτεί O(n log n) χρόνο, ενώ ο πίνακας log2 προσθέτει μόνο O(n). Επομένως, το συνολικό κόστος προεπεξεργασίας παραμένει O(n log n). Μετά την προεπεξεργασία, η τιμή $k=\lfloor \log_2(R-L+1)\rfloor$ ανακτάται απευθείας από τον πίνακα $\log_2$ σε χρόνο O(1).

Κεφάλαιο 5. Cartesian Tree of Tree

Στο κεφάλαιο αυτό παρουσιάζουμε το Cartesian Tree of Tree, μια γενίκευση της ιδέας του Cartesian Tree από πίνακες σε σταθμισμένα δέντρα. Η δομή αυτή επιτρέπει την αναγωγή ενός ερωτήματος Bottleneck Edge Query (BEQ) σε ερώτημα LCA, ακολουθώντας λογική αντίστοιχη με την αναγωγή του RMQ σε LCA μέσω του Cartesian Tree, που παρουσιάστηκε στο Κεφάλαιο 3.

5.1 Εισαγωγή και Βασική Ιδέα

Στο κεφάλαιο 3 είδαμε ότι ένα ερώτημα RMQ σε έναν πίνακα μπορεί να μετατραπεί σε ερώτημα LCA στο αντίστοιχο Cartesian Tree. Η κεντρική ιδέα του κεφαλαίου αυτού είναι ότι η ίδια προσέγγιση μπορεί να γενικευτεί από πίνακες σε δέντρα, δηλαδή ένα ερώτημα BEQ σε ένα σταθμισμένο δέντρο T μπορεί να αναχθεί σε ερώτημα LCA σε ένα κατάλληλα κατασκευασμένο Cartesian Tree of Tree. Η αναλογία των δύο προβλημάτων είναι άμεση και συνοψίζεται στον πίνακα 5.1.

RMQ	BEQ
Πίνακας A	Σταθμισμένο δέντρο T
Στοιχείο A[i]	Ακμή (u,v) με βάρος w(u,v)
Ελάχιστο στοιχείο	Ακμή με ελάχιστο βάρος
Cartesian Tree	Cartesian Tree of Tree
RMQ(i,j)= value(LCA($node_i$,$node_j$))	BEQ(u,v)= edge(LCA($leaf_u$,$leaf_v$))

Πίνακας 5.1. Αναλογία μεταξύ RMQ και BEQ

Όπως το Cartesian Tree κατασκευάζεται από έναν πίνακα τοποθετώντας το ελάχιστο στοιχείο ως ρίζα, έτσι και στο Cartesian Tree of Tree η ακμή με το ελάχιστο βάρος αντιστοιχεί στον εσωτερικό κόμβο που αποτελεί τη ρίζα της δομής. Η αφαίρεση της ακμής αυτής χωρίζει το αρχικό δέντρο σε δύο συνδεδεμένα τμήματα και η ίδια διαδικασία εφαρμόζεται αναδρομικά σε καθένα από αυτά.

5.2 Ορισμός και Ιδιότητες

Ορισμός

Έστω ένα μη κατευθυνόμενο σταθμισμένο δέντρο T με n κόμβους και n-1 ακμές. Το Cartesian Tree of Tree του T είναι ένα δυαδικό δέντρο C το οποίο ορίζεται αναδρομικά ως εξής:

- Η ρίζα του C είναι ένας εσωτερικός κόμβος που αντιστοιχεί στην ακμή e=(u,v) του T με το ελάχιστο βάρος w(e).
- Η αφαίρεση της ακμής e χωρίζει το T σε δύο συνδεδεμένα τμήματα T_L και T_R.
- Το αριστερό υποδέντρο είναι το Cartesian Tree of Tree του T_L.
- Το δεξί υποδέντρο είναι το Cartesian Tree of Tree του T_R.
- Όταν ένα τμήμα αποτελείται από έναν μόνο κόμβο και δεν περιέχει ακμές, δημιουργείται ένα φύλλο που αντιστοιχεί στον συγκεκριμένο κόμβο του αρχικού δέντρου.

Ιδιότητες

Το Cartesian Tree of Tree C ενός δέντρου T με n κόμβους και n-1 ακμές έχει τις ακόλουθες ιδιότητες:

Ιδιότητα 1- Εσωτερικοί κόμβοι: Κάθε εσωτερικός κόμβος του C αντιστοιχεί σε μία ακμή του T. Άρα το C έχει ακριβώς n-1 εσωτερικούς κόμβους.

Ιδιότητα 2- Φύλλα: Κάθε φύλλο του C αντιστοιχεί σε έναν κόμβο του T. Άρα το C έχει ακριβώς n φύλλα.

Ιδιότητα 3- Heap property: Κάθε εσωτερικός κόμβος του C αντιστοιχεί σε ακμή με βάρος μικρότερο ή ίσο από τα βάρη των ακμών που αντιστοιχούν στους εσωτερικούς απογόνους του.

Ιδιότητα 4- BEQ: Για οποιοδήποτε ζεύγος κόμβων u,v του T ισχύει: $BEQ_T(u,v)=edge(LCA_C(leaf_u,leaf_v))$, όπου $leaf_u$ και $leaf_v$ είναι τα φύλλα του C που αντιστοιχούν στους κόμβους u και v του T, ενώ edge(w) είναι η ακμή του T που αντιστοιχεί στον εσωτερικό κόμβο w.

Παράδειγμα

Έστω το σταθμισμένο δέντρο T της εικόνας 5.2 με τις ακόλουθες ακμές και βάρη:

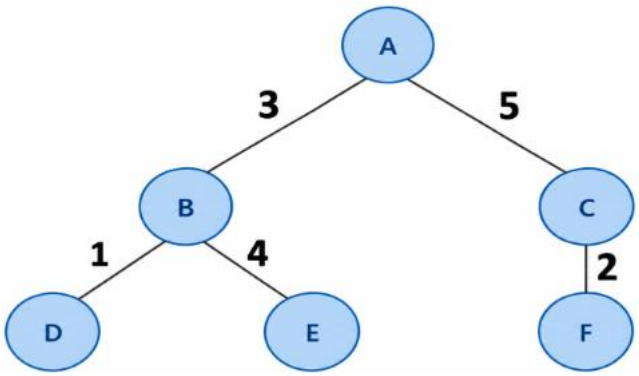
Ακμή A-B: βάρος= 3

Ακμή A-C: βάρος= 5

Ακμή B-D: βάρος= 1

Ακμή B-E: βάρος= 4

Ακμή C-F: βάρος= 2



Εικόνα 5.2. Σταθμισμένο δέντρο T

Το Cartesian Tree of Tree του T έχει τις ακόλουθες ιδιότητες:

Κόμβος του C	Τύπος	Αριστερό παιδί	Δεξί παιδί
B-D (βάρος 1)	Ρίζα	Φύλλο D	C-F (βάρος 2)
C-F (βάρος 2)	Εσωτερικός	A-B (βάρος 3)	Φύλλο F
A-B (βάρος 3)	Εσωτερικός	A-C (βάρος 5)	B-E (βάρος 4)
A-C (βάρος 5)	Εσωτερικός	Φύλλο A	Φύλλο C
B-E (βάρος 4)	Εσωτερικός	Φύλλο B	Φύλλο E

Πίνακας 5.3. Σχέσεις γονέα–παιδιού στο Cartesian Tree of Tree για το δέντρο της Εικόνας 5.2

5.3 Κατασκευή

Ο αλγόριθμος κατασκευής του Cartesian Tree of Tree βασίζεται στην ταξινόμηση των ακμών του αρχικού δέντρου και στη χρήση της δομής Union-Find. Αντί να εντοπίζουμε επαναληπτικά την ακμή ελάχιστου βάρους και να διασπούμε αναδρομικά το δέντρο, επεξεργαζόμαστε τις ακμές κατά φθίνουσα σειρά βάρους και ενώνουμε σταδιακά τις αντίστοιχες συνδεδεμένες συνιστώσες.

Αλγόριθμος Κατασκευής

1) Για κάθε κόμβο v του αρχικού δέντρου δημιουργούμε ένα φύλλο του Cartesian Tree of Tree και θεωρούμε αρχικά κάθε κόμβο ως ξεχωριστή συνιστώσα.
2) Αρχικοποιούμε τη δομή Union-Find, η οποία χρησιμοποιείται για την παρακολούθηση των συνδεδεμένων συνιστωσών.
3) Ταξινομούμε τις ακμές του αρχικού δέντρου κατά φθίνουσα σειρά βάρους.
4) Για κάθε ακμή e=(u,v), εντοπίζουμε με τη λειτουργία Find τις συνιστώσες στις οποίες ανήκουν οι κόμβοι u και v.

5) Δημιουργούμε έναν νέο εσωτερικό κόμβο του Cartesian Tree of Tree που αντιστοιχεί στην ακμή e. Ως αριστερό και δεξί παιδί του τοποθετούμε τα ήδη κατασκευασμένα υποδέντρα που αντιστοιχούν στις δύο συνιστώσες.
6) Ενώνουμε τις δύο συνιστώσες με τη λειτουργία Union και αποθηκεύουμε τον νέο εσωτερικό κόμβο ως τη ρίζα του Cartesian Tree of Tree της νέας συνιστώσας.

Μετά την επεξεργασία όλων των ακμών, απομένει μία μόνο συνιστώσα και το αντίστοιχο υποδέντρο αποτελεί το τελικό Cartesian Tree of Tree. Επειδή οι ακμές εξετάζονται κατά φθίνουσα σειρά βάρους, οι ακμές με μικρότερο βάρος επεξεργάζονται αργότερα και τοποθετούνται υψηλότερα στο τελικό δέντρο. Συνεπώς, η ακμή ελάχιστου βάρους του αρχικού δέντρου αντιστοιχεί στη ρίζα του Cartesian Tree of Tree, όπως απαιτεί ο ορισμός της δομής.

Ψευδοκώδικας

```
Algorithm BuildCartesianTreeOfTree(T)
for each vertex v of T do
   MakeSet(v)
   componentTree[v] ← new Leaf(v)
sort edges of T in decreasing order of weight
for each edge e = (u, v) in sorted order do
   rootU ← Find(u)
   rootV ← Find(v)
   newNode ← new InternalNode(e)
   newNode.left ← componentTree[rootU]
   newNode.right ← componentTree[rootV]
   newRoot ← Union(rootU, rootV)
   componentTree[newRoot] ← newNode
root ← componentTree[Find(any vertex)]
```

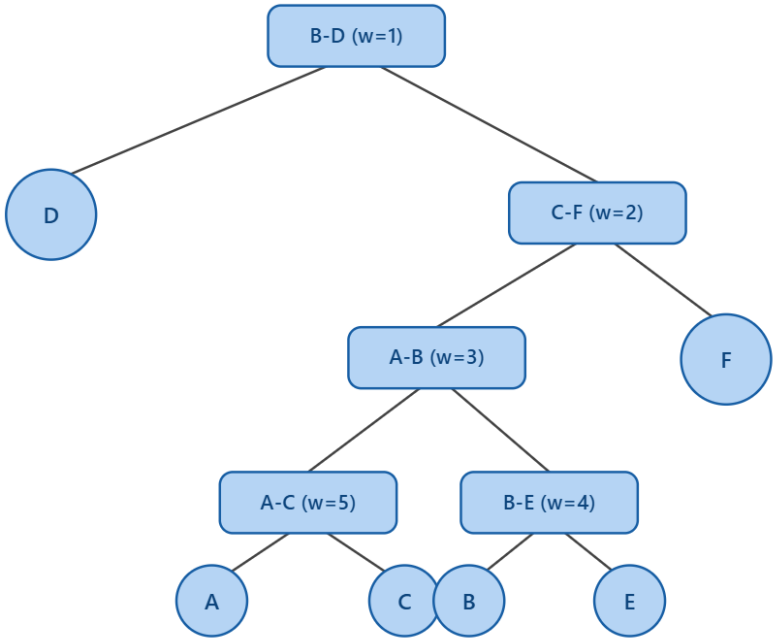
Στην υλοποίηση της εργασίας, η Union-Find χρησιμοποιεί τις τεχνικές path compression και union by rank, οι οποίες έχουν ήδη παρουσιαστεί στην Ενότητα 2.2.

Παράδειγμα βήμα προς βήμα

Έστω το σταθμισμένο δέντρο T της εικόνας 5.2, με ακμές: A-B (3), A-C (5), B-D (1), B-E (4), C-F (2). Οι ακμές ταξινομούνται κατά φθίνουσα σειρά βάρους: A-C (5), B-E (4), A-B (3), C-F (2) , B-D (1).

1) Επεξεργαζόμαστε την ακμή A-C με βάρος 5: Οι κόμβοι A και C ανήκουν σε διαφορετικές συνιστώσες. Δημιουργείται ο εσωτερικός κόμβος A-C, με παιδιά τα φύλλα A και C, και οι δύο συνιστώσες ενώνονται.
2) Επεξεργαζόμαστε την ακμή B-E με βάρος 4: Δημιουργείται εσωτερικός κόμβος B-E, με παιδιά τα φύλλα B και E, και οι δύο συνιστώσες ενώνονται.
3) Επεξεργαζόμαστε την ακμή A-B με βάρος 3: Η ακμή συνδέει πλέον τη συνιστώσα {A,C} με τη συνιστώσα {B,E}. Δημιουργείται εσωτερικός κόμβος A-B, με παιδιά τα υποδέντρα που έχουν ρίζες τους κόμβους A-C και B-E.
4) Επεξεργαζόμαστε την ακμή C-F με βάρος 2: Η ακμή συνδέει τη συνιστώσα {A,B,C,E} με τον κόμβο F. Δημιουργείται εσωτερικός κόμβος C-F, με παιδιά το ήδη κατασκευασμένο υποδέντρο και το φύλλο F.
5) Επεξεργαζόμαστε την ακμή B-D με βάρος 1: Η ακμή συνδέει την υπάρχουσα συνιστώσα με τον κόμβο D. Δημιουργείται ο εσωτερικός κόμβος B-D, ο οποίος γίνεται η ρίζα του τελικού Cartesian Tree of Tree.

Το αποτέλεσμα της κατασκευής φαίνεται στην Εικόνα 5.4, όπου οι εσωτερικοί κόμβοι αντιστοιχούν στις ακμές του αρχικού δέντρου και τα φύλλα στους αρχικούς κόμβους.



Εικόνα 5.4. Αποτέλεσμα της κατασκευής Cartesian Tree of Tree για το T

Πολυπλοκότητα

	Πολυπλοκότητα
Χρόνος κατασκευής	O(n log n)
Χώρος	O(n)

Πίνακας 5.5. Πολυπλοκότητα της υλοποίησης κατασκευής Cartesian Tree of Tree

Για ένα δέντρο με n κόμβους υπάρχουν n-1 ακμές. Η ταξινόμηση των ακμών απαιτεί χρόνο O(n log n). Για κάθε ακμή πραγματοποιείται σταθερός αριθμός πράξεων Find και Union. Με χρήση path compression και union by rank, το συνολικό κόστος των πράξεων Union-Find είναι O(nα(n)). Επομένως, επειδή η ταξινόμηση αποτελεί τον κυρίαρχο όρο, ο συνολικός χρόνος κατασκευής είναι O(n log n).

Το Cartesian Tree of Tree περιέχει n φύλλα και n-1 εσωτερικούς κόμβους, δηλαδή συνολικά 2n-1 κόμβους. Οι βοηθητικοί πίνακες της Union-Find και ο πίνακας componentTree, που αντιστοιχίζει κάθε συνιστώσα στο ήδη κατασκευασμένο υποδέντρο της, απαιτούν επίσης γραμμικό χώρο. Συνεπώς, ο συνολικός χώρος παραμένει O(n).

5.4 Απάντηση BEQ Ερωτημάτων

Σύνδεση BEQ και LCA

Όπως αναφέρθηκε και στην ενότητα 5.2, για οποιοδήποτε ζεύγος κόμβων u,v του αρχικού δέντρου T ισχύει:

$$BEQ_T(u,v)=edge(LCA_C(leaf_u,leaf_v))$$

Για την απάντηση ενός ερωτήματος BEQ(u,v), εντοπίζουμε αρχικά τα φύλλα $leaf_u$ και $leaf_v$ στο Cartesian Tree of Tree. Στη συνέχεια υπολογίζουμε τον ελάχιστο κοινό πρόγονό τους και επιστρέφουμε την ακμή που αντιστοιχεί στον συγκεκριμένο εσωτερικό κόμβο.

Στην online υλοποίηση της εργασίας χρησιμοποιείται η απλή μέθοδος LCA πάνω στο Cartesian Tree of Tree. Η κλάση TreeQuery εντοπίζει τα δύο αντίστοιχα φύλλα και στη συνέχεια καλεί τη διαδικασία LCA για τον υπολογισμό του κοινού προγόνου.

Ψευδοκώδικας

Algorithm BEQ(u,v,C)

$leaf_u \leftarrow$FindLeaf(C,u)

$leaf_v \leftarrow$FindLeaf(C,v)

lcaNode$\leftarrow$ LCA($leaf_u$,$leaf_v$)

return edge(lcaNode)

όπου FindLeaf(C,u) βρίσκει το φύλλο του C που αντιστοιχεί στον κόμβο u του T, και edge(lcaNode) επιστρέφει την ακμή του T που αντιστοιχεί στον εσωτερικό κόμβο lcaNode.

Παράδειγμα

Έστω το Cartesian Tree of Tree C της εικόνας 5.4. Εξετάζουμε τα ακόλουθα ερωτήματα:

Ερώτημα 1: BEQ(D,F)

Μονοπάτι D →B →A →C →F

Φύλλο D → μονοπάτι προς ρίζα: D → B-D (1)

Φύλλο F → μονοπάτι προς ρίζα: F → C-F (2) → B-D (1)

LCA(D, F)= B-D (1)

BEQ(D,F)= ακμή B-D (βάρος=1)

Ερώτημα 2: BEQ(D,E)

Μονοπάτι D →B →E

Φύλλο D → μονοπάτι προς ρίζα: D → B-D (1)

Φύλλο E → μονοπάτι προς ρίζα: E →B-E (4) →A-B (3) → C-F (2) → B-D (1)

LCA(D, E)= B-D (1)

BEQ(D,E)= ακμή B-D (βάρος=1)

Ερώτημα 3: BEQ(E,F)

Μονοπάτι E →B →A →C →F

Φύλλο E → μονοπάτι προς ρίζα: E → B-E (4) → A-B (3) → C-F (2) → B-D (1)

Φύλλο F → μονοπάτι προς ρίζα: F → C-F (2) → B-D (1)

LCA(E, F)= C-F (2)

BEQ(E,F)= ακμή C-F (βάρος=2)

Παρατηρούμε ότι σε κάθε περίπτωση ο LCA στο Cartesian Tree of Tree αντιστοιχεί στην ακμή με το ελάχιστο βάρος στο μονοπάτι μεταξύ των δύο κόμβων στο αρχικό δέντρο, επιβεβαιώνοντας στα συγκεκριμένα παραδείγματα τη σχέση μεταξύ BEQ και LCA μέσω του Cartesian Tree of Tree.

Offline απάντηση BEQ ερωτημάτων

Όταν όλα τα BEQ ερωτήματα είναι γνωστά εκ των προτέρων, μπορούν να μετατραπούν σε LCA ερωτήματα πάνω στο Cartesian Tree of Tree και να επιλυθούν από κοινού με τον Tarjan Offline LCA.

Για κάθε κόμβο u του αρχικού δέντρου χρησιμοποιείται το αντίστοιχο φύλλο $leaf_u$ του Cartesian Tree of Tree. Επομένως, κάθε ερώτημα BEQ(u,v) μετατρέπεται στο ερώτημα $LCA_C(leaf_u,leaf_v)$.

Ο Tarjan Offline LCA εκτελεί μία διάσχιση DFS του Cartesian Tree of Tree. Κατά τη διάρκεια της διάσχισης, κάθε κόμβος αρχικοποιείται ως ξεχωριστό σύνολο στη δομή Union-Find. Αφού ολοκληρωθεί η επεξεργασία ενός παιδιού, το σύνολό του ενώνεται με το σύνολο του γονέα και ο κατάλληλος πρόγονος αποθηκεύεται στη δομή ancestor για τον εκπρόσωπο της νέας συνιστώσας.

Όταν κατά τη διάσχιση έχουν ήδη ολοκληρωθεί και τα δύο άκρα ενός προκαθορισμένου ερωτήματος, ο LCA τους προσδιορίζεται μέσω της τιμής ancestor[Find(x)] της αντίστοιχης Union-Find συνιστώσας. Ο εσωτερικός κόμβος του Cartesian Tree of Tree που προκύπτει αντιστοιχεί σε μία ακμή του αρχικού δέντρου και αυτή η ακμή αποτελεί την απάντηση του BEQ.

Ψευδοκώδικας

Algorithm OfflineBEQ(C, Queries)

```
for each query BEQ(u, v) do
   leafu ← leaf corresponding to u
   leafv ← leaf corresponding to v
   add LCA query (leafu, leafv)
Tarjan(C.root)
for each query q do
   lcaNode ← LCA answer of q
   answer[q] ← edge(lcaNode)
return answer
```

Η βασική διαδικασία Tarjan πάνω στο Cartesian Tree of Tree μπορεί να αποδοθεί συνοπτικά ως εξής:

Algorithm Tarjan(u)

```
MakeSet(u)
ancestor[Find(u)] ← u
for each child v of u do
   Tarjan(v)
   Union(u, v)
   ancestor[Find(u)] ← u
mark u as finished
for each query (u, x) associated with u do
   if x is finished then
      lca ← ancestor[Find(x)]
      store lca as answer of the query
```

Παράδειγμα offline επεξεργασίας

Έστω ότι είναι γνωστά εκ των προτέρων τα εξής δύο ερωτήματα: BEQ(D,F) και BEQ(E,F). Στο Cartesian Tree of Tree τα ερωτήματα μετατρέπονται σε $LCA_C(leaf_D,leaf_F)$ και $LCA_C(leaf_E,leaf_F)$. Κατά τη DFS διάσχιση του CToT, οι συνιστώσες των φύλλων ενώνονται σταδιακά με τις συνιστώσες των γονέων τους.

Για το πρώτο ερώτημα, όταν έχουν ολοκληρωθεί τα $leaf_D$ και $leaf_F$, ο Tarjan επιστρέφει ως κοινό πρόγονο τον εσωτερικό κόμβο B-D(1). Επομένως BEQ(D,F)=B-D με βάρος 1.

Για το δεύτερο ερώτημα, ο κοινός πρόγονος των $leaf_E$ και $leaf_F$ είναι ο εσωτερικός κόμβος C-F(2). Επομένως BEQ(E,F)=C-F με βάρος 2.

Το βασικό σημείο είναι ότι τα ερωτήματα δεν απαντώνται ένα προς ένα με ξεχωριστές αναζητήσεις στο δέντρο. Είναι όλα γνωστά πριν από τη διάσχιση και απαντώνται κατά τη διάρκεια της ίδιας DFS εκτέλεσης του Tarjan.

Πολυπλοκότητα

Λειτουργία	Χρόνος
Κατασκευή Cartesian Tree of Tree	O(n log n)
Online BEQ ερώτημα με απλή LCA	O(n)
Offline επεξεργασία q BEQ ερωτημάτων	O((n+q)α(n))

Πίνακας 5.6. Πολυπλοκότητα της υλοποίησης για την απάντηση BEQ ερωτημάτων

Η κατασκευή του Cartesian Tree of Tree απαιτεί O(n log n) χρόνο, όπως αναλύθηκε στην Ενότητα 5.3. Στην online υλοποίηση, για κάθε ερώτημα BEQ(u,v) εντοπίζονται αρχικά τα φύλλα που αντιστοιχούν στους κόμβους u και v. Η αναζήτηση κάθε φύλλου μπορεί στη χειρότερη περίπτωση να απαιτήσει διάσχιση ολόκληρου του Cartesian Tree of Tree και επομένως χρόνο O(n). Στη συνέχεια υπολογίζεται ο LCA των δύο φύλλων με μετακίνηση προς τους γονείς τους, η οποία επίσης μπορεί στη χειρότερη περίπτωση να απαιτήσει O(n) χρόνο. Συνεπώς, κάθε online BEQ ερώτημα απαντάται σε O(n) χρόνο.

Στην offline περίπτωση, όταν όλα τα q ερωτήματα είναι γνωστά εκ των προτέρων, χρησιμοποιείται ο Tarjan Offline LCA. Για ένα Cartesian Tree of Tree με 2n-1 κόμβους και q ερωτήματα, η χρήση Union-Find με path compression και union by rank οδηγεί σε συνολικό χρόνο O((n+q)α(n)) για την επεξεργασία των ερωτημάτων.

Κεφάλαιο 6. Πειραματική Μελέτη

Στο κεφάλαιο αυτό παρουσιάζουμε την πειραματική μελέτη των υλοποιήσεων που περιγράφηκαν στα προηγούμενα κεφάλαια. Σκοπός της αξιολόγησης είναι διττός. Αφενός, εξετάζουμε τη συμφωνία των αποτελεσμάτων των υλοποιήσεων με τις αντίστοιχες απλοϊκές λύσεις αναφοράς και, αφετέρου, μελετούμε τη συμπεριφορά των δομών δεδομένων σε διαφορετικές περιπτώσεις εισόδου, οι οποίες αναδεικνύουν ιδιαίτερα χαρακτηριστικά τους.

6.1 Περιγραφή Υλοποίησης

Η υλοποίηση πραγματοποιήθηκε στη γλώσσα προγραμματισμού Java και οργανώνεται στις ακόλουθες κλάσεις:

- Για το πρόβλημα RMQ:
  - CartesianTree: Δέχεται έναν πίνακα A από n ακέραιους αριθμούς και κατασκευάζει το Cartesian Tree σε γραμμικό χρόνο με χρήση στοίβας. Τα στοιχεία του πίνακα εξετάζονται από αριστερά προς τα δεξιά και οι σχέσεις αριστερού και δεξιού παιδιού ενημερώνονται κατάλληλα ώστε να διατηρούνται η ιδιότητα σωρού και η inorder διάταξη.
  - CartesianTreeNode: Αναπαριστά έναν κόμβο του Cartesian Tree και αποθηκεύει την τιμή του αντίστοιχου στοιχείου του πίνακα, τη θέση του και αναφορές στα δύο παιδιά του.
  - LCA: Δέχεται το Cartesian Tree και κατασκευάζει μία βοηθητική δομή για τον υπολογισμό LCA, αποθηκεύοντας για κάθε κόμβο το βάθος και τον γονέα του. Για κάθε ερώτημα, οι δύο κόμβοι μετακινούνται προς τους γονείς τους μέχρι να βρεθεί ο ελάχιστος κοινός πρόγονος. Τα ερωτήματα RMQ απαντώνται μέσω του LCA των κόμβων που αντιστοιχούν στις ζητούμενες θέσεις.
  - LCANode: Χρησιμοποιείται από την απλή υλοποίηση LCA για την αποθήκευση των πληροφοριών που απαιτούνται για κάθε κόμβο κατά την επεξεργασία των ερωτημάτων.

- OptimalLCA: Υλοποιεί μία static-preprocessed προσέγγιση για την απάντηση LCA ερωτημάτων. Πραγματοποιεί Euler tour του Cartesian Tree και χρησιμοποιεί το αντίστοιχο πρόβλημα RMQ πάνω στον πίνακα των βαθών, εκμεταλλευόμενη την ιδιότητα ότι διαδοχικά βάθη διαφέρουν κατά +1 ή -1. Η θεωρητική μέθοδος στην οποία βασίζεται επιτυγχάνει O(n) προεπεξεργασία και O(1) χρόνο ανά ερώτημα.
- OfflineLCA: Υλοποιεί τον Tarjan Offline LCA για την περίπτωση όπου όλα τα ερωτήματα είναι γνωστά εκ των προτέρων. Τα ερωτήματα RMQ μετατρέπονται σε LCA ερωτήματα πάνω στο Cartesian Tree και απαντώνται από κοινού με χρήση DFS και Union-Find με path compression και union by rank.
- SparseTable: Κατασκευάζει τον πίνακα st[i][j] σε χρόνο O(n log n) και απαντά κάθε ερώτημα RMQ(L,R) σε χρόνο O(1), χρησιμοποιώντας δύο ενδεχομένως επικαλυπτόμενα διαστήματα μήκους δύναμης του 2.
- NaiveRMQ: Υλοποιεί τη γραμμική αναζήτηση του ελάχιστου στοιχείου μέσα στο ζητούμενο διάστημα και χρησιμοποιείται ως λύση αναφοράς για τη σύγκριση των αποτελεσμάτων των υπόλοιπων μεθόδων.

➢ Για το πρόβλημα BEQ:

- CartesianTreeOfTree: Δέχεται ένα σταθμισμένο δέντρο T με n κόμβους και n-1 ακμές και κατασκευάζει το Cartesian Tree of Tree με ταξινόμηση των ακμών κατά φθίνουσα σειρά βάρους και χρήση Union-Find. Κάθε αρχικός κόμβος αντιστοιχεί αρχικά σε ένα φύλλο και, καθώς επεξεργάζονται οι ακμές, δημιουργούνται εσωτερικοί κόμβοι που συνδέουν τα υποδέντρα των αντίστοιχων συνιστωσών. Η υλοποίηση χρησιμοποιεί path compression και union by rank.
- CartesianTreeOfTreeNode: Αναπαριστά έναν κόμβο του Cartesian Tree of Tree. Ένας κόμβος μπορεί να είναι είτε φύλλο που αντιστοιχεί σε κόμβο του αρχικού δέντρου είτε εσωτερικός κόμβος που αντιστοιχεί σε ακμή.
- CartesianTreeOfTreeEdge: Αναπαριστά μία ακμή του αρχικού σταθμισμένου δέντρου, αποθηκεύοντας τα δύο άκρα και το βάρος της.

- CartesianTreeOfTreeLCA: Κατασκευάζει τη βοηθητική πληροφορία βάθους και γονέα για τους κόμβους του Cartesian Tree of Tree και υποστηρίζει την απλή online μέθοδο LCA. Επίσης παρέχει τη διαδικασία findLeaf για τον εντοπισμό του φύλλου που αντιστοιχεί σε έναν κόμβο του αρχικού δέντρου.
- TreeQuery: Δέχεται δύο κόμβους u και v του αρχικού δέντρου, εντοπίζει τα αντίστοιχα φύλλα $leaf_u$ και $leaf_v$ στο Cartesian Tree of Tree, υπολογίζει τον LCA($leaf_u$,$leaf_v$) και επιστρέφει την ακμή του αρχικού δέντρου που αντιστοιχεί στον συγκεκριμένο εσωτερικό κόμβο.
- OfflineCToTLCA: Υλοποιεί την offline απάντηση BEQ ερωτημάτων με βάση τον Tarjan Offline LCA πάνω στο Cartesian Tree of Tree. Τα προκαθορισμένα BEQ ερωτήματα μετατρέπονται σε LCA ερωτήματα μεταξύ των αντίστοιχων φύλλων και οι απαντήσεις υπολογίζονται από κοινού με χρήση Union-Find.
- NaiveBEQ: Εντοπίζει το μοναδικό μονοπάτι μεταξύ δύο κόμβων του αρχικού δέντρου χρησιμοποιώντας BFS και στη συνέχεια βρίσκει την ακμή με το ελάχιστο βάρος πάνω στο μονοπάτι. Χρησιμοποιείται ως λύση αναφοράς για την πειραματική επαλήθευση της συμφωνίας των αποτελεσμάτων.

➢ Κεντρική κλάση εκτέλεσης: Main: Αποτελεί την κεντρική κλάση εκτέλεσης της εφαρμογής και συντονίζει την πειραματική αξιολόγηση των υλοποιήσεων. Διαβάζει από τα αρχεία εισόδου τους πίνακες, τα σταθμισμένα δέντρα και τα αντίστοιχα ερωτήματα, δημιουργεί τις απαραίτητες δομές δεδομένων και εκτελεί τις διαφορετικές μεθόδους επίλυσης. Για το RMQ συγκρίνει τα αποτελέσματα των NaiveRMQ, SparseTable, LCA, OptimalLCA και OfflineLCA, ενώ για το BEQ συγκρίνει τις TreeQuery, NaiveBEQ και την offline μέθοδο που βασίζεται στον Tarjan Offline LCA. Με αυτόν τον τρόπο ελέγχεται η συμφωνία των αποτελεσμάτων των διαφορετικών υλοποιήσεων στα test cases που χρησιμοποιούνται στην πειραματική μελέτη.

Τα δεδομένα εισόδου διαβάζονται από αρχεία κειμένου.

Τα **αρχεία πινάκων** έχουν την ακόλουθη μορφή: Η πρώτη γραμμή περιέχει το πλήθος n των στοιχείων και η δεύτερη γραμμή τα στοιχεία του πίνακα χωρισμένα με κενά.

Τα **αρχεία δέντρων** έχουν την ακόλουθη μορφή: Η πρώτη γραμμή περιέχει το πλήθος n των κόμβων και το πλήθος m των ακμών, και κάθε επόμενη γραμμή περιέχει τους δύο κόμβους και το βάρος μίας ακμής. Στην περίπτωση των δέντρων ισχύει m= n-1.

Τα **αρχεία ερωτημάτων** έχουν την ακόλουθη μορφή: Η πρώτη γραμμή περιέχει το πλήθος q των ερωτημάτων και κάθε επόμενη γραμμή ένα ζεύγος τιμών. Για τα ερωτήματα RMQ οι δύο τιμές αντιστοιχούν σε δείκτες του πίνακα, ενώ για τα ερωτήματα BEQ αντιστοιχούν σε κόμβους του αρχικού δέντρου.

6.2 Test Cases για RMQ

Για την αξιολόγηση των υλοποιήσεων RMQ σχεδιάσαμε πέντε διαφορετικούς πίνακες, οι οποίοι καλύπτουν τόσο τυπικές όσο και ακραίες περιπτώσεις εισόδου. Οι συγκεκριμένες περιπτώσεις επιλέχθηκαν ώστε να εξεταστεί η συμπεριφορά των διαφορετικών μεθόδων σε πίνακες που οδηγούν σε Cartesian Trees διαφορετικής μορφής και ύψους, καθώς και σε περιπτώσεις με ίσες τιμές ή έντονα μη ισορροπημένη δομή.

- **array1:** Μικρό παράδειγμα (n=5)
  Ο πίνακας A=[3,2,6,1,9] χρησιμοποιείται ως βασικό παράδειγμα ελέγχου και έχει ήδη χρησιμοποιηθεί στα προηγούμενα κεφάλαια. Το Cartesian Tree που προκύπτει έχει ρίζα τον κόμβο με τιμή 1 (θέση 3) και μέγιστο βάθος 2.
- **array2:** Αύξουσα διάταξη (n=50)
  Ο πίνακας A=[1,2,3, ... ,50] οδηγεί σε εκφυλισμένο Cartesian Tree προς τα δεξιά, καθώς κάθε νέο στοιχείο τοποθετείται ως δεξί παιδί του προηγούμενου. Το μέγιστο βάθος είναι n-1=49. Η περίπτωση αυτή είναι χρήσιμη για την αξιολόγηση των μεθόδων LCA σε ένα ιδιαίτερα μη ισορροπημένο δέντρο. Εκτελούμε 15 ερωτήματα RMQ που καλύπτουν διαστήματα διαφορετικού μεγέθους και θέσης.
- **array3:** Φθίνουσα διάταξη (n=50)
  Ο πίνακας A=[50,49,48, ... ,1] οδηγεί επίσης σε εκφυλισμένο Cartesian Tree, αυτή τη φορά προς τα αριστερά, με μέγιστο βάθος n-1=49. Η περίπτωση αυτή είναι συμμετρική της array2 και επιτρέπει την εξέταση της συμπεριφοράς των υλοποιήσεων σε ένα δεύτερο ακραίο σχήμα δέντρου. Εκτελούμε 15 ερωτήματα RMQ.

- **array4:** Τυχαία στοιχεία (n=50)
  Ο πίνακας A=[42,7,35,14,28,3,...] αποτελεί περίπτωση τυχαίας εισόδου. Το Cartesian Tree που προκύπτει έχει μέγιστο βάθος 8, σημαντικά μικρότερο από το βάθος 49 των εκφυλισμένων περιπτώσεων array2, array3 και array5. Η περίπτωση αυτή παρουσιάζει μία περισσότερο ισορροπημένη μορφή του Cartesian Tree και μας επιτρέπει να εξετάσουμε τη συμπεριφορά της υλοποίησης σε μία πιο τυπική είσοδο. Εκτελούμε 15 ερωτήματα RMQ που καλύπτουν διαστήματα σε διαφορετικές περιοχές του πίνακα.
- **array5**: Ίδια στοιχεία (n=50)
  Ο πίνακας A=[5,5,5, ... ,5] εξετάζει τη συμπεριφορά της υλοποίησης όταν όλα τα στοιχεία έχουν την ίδια τιμή. Στη συγκεκριμένη υλοποίηση, η αυστηρή σύγκριση > διατηρεί την πρώτη εμφάνιση μεταξύ ίσων ελάχιστων τιμών. Επομένως, το Cartesian Tree εκφυλίζεται σε γραμμική δομή προς τα δεξιά, όπως και στο array2, με μέγιστο βάθος 49. Η περίπτωση αυτή εξετάζει ειδικά τη συμπεριφορά της υλοποίησης όταν υπάρχουν πολλαπλές ίσες ελάχιστες τιμές. Εκτελούμε 15 ερωτήματα RMQ.

Test	Μέγεθος	Περιγραφή	Μορφή Cartesian Tree	Ερωτήματα
array1	n=5	Μικρό παράδειγμα	Μέγιστο βάθος 2	5
array2	n=50	Αύξουσα διάταξη	Γραμμική δομή προς τα δεξιά (βάθος 49)	15
array3	n=50	Φθίνουσα διάταξη	Γραμμική δομή προς τα αριστερά (βάθος 49)	15
array4	n=50	Τυχαία σειρά	Μέγιστο βάθος 8	15
array5	n=50	Ίσα στοιχεία	Γραμμική δομή προς τα δεξιά (βάθος 49)	15

Πίνακας 6.1. Test cases για RMQ

6.3 Test Cases για BEQ

Για την αξιολόγηση του BEQ σχεδιάσαμε τέσσερα δέντρα διαφορετικής τοπολογίας και μεγέθους. Ο λόγος που επιλέξαμε αυτές τις συγκεκριμένες τοπολογίες είναι ότι η συμπεριφορά του Cartesian Tree of Tree εξαρτάται από τη δομή του αρχικού δέντρου και τη διάταξη των βαρών των ακμών. Για κάθε test case εκτελούνται προκαθορισμένα ερωτήματα BEQ, τα αποτελέσματα των οποίων χρησιμοποιούνται για

τη σύγκριση της online μεθόδου μέσω του Cartesian Tree of Tree, της offline μεθόδου με Tarjan LCA και της απλοϊκής λύσης αναφοράς NaiveBEQ.

- **tree1:** Μικρό διακλαδωμένο δέντρο (n=5)

  Το δέντρο με 5 κόμβους και 4 ακμές χρησιμοποιείται ως βασικό παράδειγμα επαλήθευσης. Οι ακμές είναι 0-1 (βάρος 4), 0-2 (βάρος 1), 0-3 (βάρος 5), 2-4 (βάρος 3). Με ρίζα τον κόμβο 0, τρεις κόμβοι συνδέονται άμεσα με τη ρίζα, ενώ ο κόμβος 4 βρίσκεται σε βάθος 2 μέσω του κόμβου 2. Το ύψος του δέντρου είναι επομένως 2. Το Cartesian Tree of Tree που προκύπτει έχει ως ρίζα την ακμή 0-2 η οποία έχει το ελάχιστο βάρος 1 και μέγιστο βάθος 3. Εκτελούμε 6 ερωτήματα BEQ που καλύπτουν ζεύγη κόμβων σε διαφορετικά μονοπάτια.
- **tree2:** Γραμμικό δέντρο (n=30)

  Το γραμμικό δέντρο με 30 κόμβους αποτελεί μία αλυσίδα κόμβων 0-1-2-... -29. Κάθε κόμβος, εκτός από τα δύο άκρα, έχει βαθμό 2. Θεωρώντας ως ρίζα τον κόμβο 0, ο κόμβος 29 βρίσκεται σε βάθος 29 και συνεπώς το ύψος του αρχικού δέντρου είναι 29. Τα βάρη των ακμών λαμβάνουν επαναλαμβανόμενες τιμές από 1 έως 9, με αποτέλεσμα να εμφανίζονται ακμές ελάχιστου βάρους σε διαφορετικές θέσεις της αλυσίδας. Η συγκεκριμένη τοπολογία επιτρέπει την επαλήθευση της συμπεριφοράς του αλγορίθμου σε περιπτώσεις όπου το μονοπάτι μεταξύ δύο κόμβων περιλαμβάνει μεγάλο αριθμό ακμών. Το Cartesian Tree of Tree που προκύπτει έχει μέγιστο βάθος 7. Εκτελούμε 10 ερωτήματα BEQ που καλύπτουν ζεύγη κόμβων που βρίσκονται σε διαφορετικές και συχνά απομακρυσμένες θέσεις της αλυσίδας.
- **tree3:** Δέντρο τύπου αστέρα (n=25)

  Το δέντρο αστέρα με 25 κόμβους αποτελείται από έναν κεντρικό κόμβο 0 που συνδέεται με τους υπόλοιπους 24 κόμβους. Θεωρώντας ως ρίζα τον κόμβο 0, όλοι οι υπόλοιποι κόμβοι βρίσκονται σε βάθος 1 και επομένως το ύψος του δέντρου είναι 1. Τα βάρη των ακμών κυμαίνονται από 1 ως 10. Η συγκεκριμένη τοπολογία οδηγεί σε ένα έντονα μη ισορροπημένο Cartesian Tree of Tree, στο οποίο οι εσωτερικοί κόμβοι σχηματίζουν σχεδόν γραμμική αλυσίδα. Το μέγιστο βάθος του Cartesian Tree of Tree είναι 24, συμπεριλαμβανομένου του επιπέδου των φύλλων. Επιπλέον, κάθε μονοπάτι μεταξύ δύο μη κεντρικών κόμβων διέρχεται από τον κεντρικό κόμβο 0, επομένως το αποτέλεσμα ενός ερωτήματος BEQ καθορίζεται από

το ελάχιστο βάρος των δύο ακμών του μονοπατιού. Για το συγκεκριμένο test case εκτελούνται 10 ερωτήματα BEQ.

- **tree4:** Πλήρες δυαδικό δέντρο (n=63)
  Το τέταρτο test case αποτελείται από 63 κόμβους και 62 ακμές και έχει τη μορφή πλήρους δυαδικού δέντρου. Με ρίζα τον κόμβο 0, κάθε εσωτερικός κόμβος έχει δύο παιδιά και οι βαθύτεροι κόμβοι βρίσκονται σε βάθος 5. Επομένως το ύψος του αρχικού δέντρου είναι 5. Τα βάρη των ακμών κυμαίνονται από 1 έως 9. Για τη συγκεκριμένη τοπολογία και διάταξη βαρών, το Cartesian Tree of Tree που προκύπτει έχει μέγιστο βάθος 14, σε αντίθεση με το tree3, όπου οι εσωτερικοί κόμβοι του Cartesian Tree of Tree σχηματίζουν σχεδόν γραμμική αλυσίδα. Για το συγκεκριμένο test case εκτελούμε 12 ερωτήματα BEQ, τα οποία περιλαμβάνουν ζεύγη κόμβων που βρίσκονται σε διαφορετικά επίπεδα και διαφορετικούς κλάδους του αρχικού δέντρου.

Test	Κόμβοι	Ακμές	Τοπολογία αρχικού δέντρου	Ύψος αρχικού δέντρου	Ύψος Cartesian Tree of Tree	Ερωτήματα
tree1	5	4	Μικρό διακλαδωμένο	2	3	6
tree2	30	29	Γραμμικό	29	7	10
tree3	25	24	Αστέρας	1	24	10
tree4	63	62	Πλήρες Δυαδικό	5	14	12

Πίνακας 6.2. Test cases για BEQ

6.4 Αποτελέσματα RMQ

Για κάθε test case εκτελούμε πέντε διαφορετικές προσεγγίσεις και συγκρίνουμε τα αποτελέσματά τους. Συγκεκριμένα, χρησιμοποιούμε τη NaiveRMQ ως λύση αναφοράς, το SparseTable, την προσέγγιση Cartesian Tree + LCA, την OptimalLCA και την offline προσέγγιση που βασίζεται στον Tarjan Offline LCA. Ο Πίνακας 6.3 παρουσιάζει ορισμένα χαρακτηριστικά ερωτήματα και τα αποτελέσματά τους.

Test	Ερώτημα	NaiveRMQ	Sparse Table	CT+LCA	Optimal LCA	Offline LCA	Match
array1	RMQ(0,4)	1	1	1	1	1	YES
array1	RMQ(0,2)	2	2	2	2	2	YES
array2	RMQ(0,49)	1	1	1	1	1	YES
array2	RMQ(10,30)	11	11	11	11	11	YES
array3	RMQ(0,49)	1	1	1	1	1	YES
array3	RMQ(15,40)	10	10	10	10	10	YES
array4	RMQ(0,49)	3	3	3	3	3	YES
array4	RMQ(7,35)	6	6	6	6	6	YES
array5	RMQ(0,49)	5	5	5	5	5	YES
array5	RMQ(20,45)	5	5	5	5	5	YES

Πίνακας 6.3. Χαρακτηριστικά αποτελέσματα ερωτημάτων RMQ

Παρατηρούμε ότι και οι πέντε προσεγγίσεις δίνουν τα ίδια αποτελέσματα σε όλα τα εξεταζόμενα ερωτήματα. Η συμφωνία των αποτελεσμάτων του Sparse Table, της προσέγγισης Cartesian Tree + LCA, της OptimalLCA και της offline μεθόδου με τη λύση αναφοράς NaiveRMQ παρέχει πειραματική επαλήθευση της ορθής λειτουργίας των υλοποιήσεων στα συγκεκριμένα test cases.

Στο array2, το οποίο είναι ταξινομημένο σε αύξουσα σειρά, το ελάχιστο κάθε εξεταζόμενου υποδιαστήματος βρίσκεται στο αριστερό άκρο του. Για παράδειγμα, το RMQ(0,49) επιστρέφει 1, ενώ το RMQ(10,30) επιστρέφει 11, καθώς το αντίστοιχο υποδιάστημα περιέχει τις τιμές [11,12, … ,31].

Στο array3, το οποίο είναι ταξινομημένο σε φθίνουσα σειρά, το ελάχιστο κάθε υποδιαστήματος βρίσκεται στο δεξί άκρο του. Για παράδειγμα, το RMQ(15,40) επιστρέφει 10, καθώς το αντίστοιχο υποδιάστημα περιέχει φθίνουσες τιμές και η μικρότερη τιμή βρίσκεται στη δεξιότερη θέση του διαστήματος. Αντίστοιχα, το RMQ(0,49) επιστρέφει 1, το οποίο είναι το τελευταίο στοιχείο του πίνακα. Η περίπτωση αυτή εξετάζει τη συμπεριφορά των υλοποιήσεων όταν το ελάχιστο βρίσκεται στο δεξί άκρο του εξεταζόμενου διαστήματος.

Στο array4, το οποίο περιέχει στοιχεία σε τυχαία διάταξη, τα αποτελέσματα εξαρτώνται από το εξεταζόμενο διάστημα. Το RMQ(0,49) επιστρέφει 3, το οποίο είναι το ελάχιστο ολόκληρου του πίνακα, ενώ το RMQ(7,35) επιστρέφει 6, το οποίο είναι το ελάχιστο του αντίστοιχου υποδιαστήματος. Η περίπτωση αυτή εξετάζει τη συμπεριφορά των υλοποιήσεων σε μία πιο τυπική μη ταξινομημένη διάταξη δεδομένων.

Στο array5, όλα τα στοιχεία έχουν τιμή 5 και επομένως κάθε ερώτημα RMQ επιστρέφει 5, ανεξάρτητα από τα όρια του διαστήματος. Η περίπτωση αυτή εξετάζει ειδικά τη συμπεριφορά των υλοποιήσεων όταν υπάρχουν πολλαπλές ίσες ελάχιστες τιμές.

Συνολικά εκτελέστηκαν 65 ερωτήματα RMQ. Σε όλα τα ερωτήματα, τα αποτελέσματα των SparseTable, Cartesian Tree + LCA, OptimalLCA και OfflineLCA συμφώνησαν με τα αποτελέσματα της λύσης αναφοράς NaiveRMQ.

6.5 Αποτελέσματα BEQ

Για κάθε test case εκτελούμε τρεις προσεγγίσεις και συγκρίνουμε τα αποτελέσματά τους: τη μέθοδο TreeQuery, η οποία χρησιμοποιεί το Cartesian Tree of Tree σε συνδυασμό με LCA, την offline μέθοδο που βασίζεται στον Tarjan Offline LCA και τη λύση αναφοράς NaiveBEQ, η οποία εντοπίζει το μονοπάτι μεταξύ των δύο κόμβων μέσω BFS. Ο Πίνακας 6.4 παρουσιάζει ορισμένα χαρακτηριστικά ερωτήματα και τα αποτελέσματά τους.

Test	Ερώτημα	TreeQuery	Offline LCA	NaiveBEQ	Match
tree1	BEQ(1,4)	ακμή 0-2 (w=1)	ακμή 0-2 (w=1)	ακμή 0-2 (w=1)	YES
tree1	BEQ(0,3)	ακμή 0-3 (w=5)	ακμή 0-3 (w=5)	ακμή 0-3 (w=5)	YES
tree2	BEQ(0,29)	ακμή 2-3 (w=1)	ακμή 2-3 (w=1)	ακμή 2-3 (w=1)	YES
tree2	BEQ(3,26)	ακμή 13-14 (w=1)	ακμή 13-14 (w=1)	ακμή 13-14 (w=1)	YES
tree3	BEQ(1,24)	ακμή 0-1 (w=5)	ακμή 0-1 (w=5)	ακμή 0-1 (w=5)	YES
tree3	BEQ(4,21)	ακμή 0-4 (w=1)	ακμή 0-4 (w=1)	ακμή 0-4 (w=1)	YES
tree4	BEQ(0,62)	ακμή 0-2 (w=1)	ακμή 0-2 (w=1)	ακμή 0-2 (w=1)	YES
tree4	BEQ(2,61)	ακμή 2-6 (w=5)	ακμή 2-6 (w=5)	ακμή 2-6 (w=5)	YES

Πίνακας 6.4. Χαρακτηριστικά αποτελέσματα ερωτημάτων BEQ

Παρατηρούμε ότι και οι τρεις προσεγγίσεις δίνουν τα ίδια αποτελέσματα σε όλα τα εξεταζόμενα ερωτήματα. Η συμφωνία των αποτελεσμάτων της TreeQuery και της offline μεθόδου με τη λύση αναφοράς NaiveBEQ αποτελεί πειραματική ένδειξη της ορθής λειτουργίας των υλοποιήσεων στα συγκεκριμένα test cases.

Στο tree1 (μικρό διακλαδωμένο), το BEQ(1,4)= ακμή 0-2 (βάρος 1) καθώς το μονοπάτι 1-0-2-4 περιέχει τις ακμές 0-1 (βάρος 4), 0-2 (βάρος 1) και 2-4 (βάρος 3), με ακμή ελάχιστου βάρους την 0-2. Αντίθετα, το BEQ(0,3)= ακμή 0-3 (βάρος 5) καθώς το μονοπάτι 0→3 αποτελείται από μία μόνο ακμή.

Στο tree2 (γραμμικό δέντρο), τα ερωτήματα BEQ(0,29), BEQ(1,28) και BEQ(2,27) επιστρέφουν όλα την ίδια ακμή 2-3 (βάρος 1), καθώς αυτή η ακμή βρίσκεται σε όλα τα μονοπάτια που διασχίζουν ολόκληρη την αλυσίδα και έχει το ελάχιστο βάρος. Αντίθετα, το BEQ(3,26) επιστρέφει την ακμή 13-14 (βάρος 1), καθώς το μονοπάτι 3→...→26 δεν περνά από την ακμή 2-3 (το αρχικό τμήμα της αλυσίδας βρίσκεται εκτός του μονοπατιού).

Στο tree3 (δέντρο τύπου αστέρα), κάθε μονοπάτι μεταξύ δύο μη κεντρικών κόμβων u και v διέρχεται από τον κεντρικό κόμβο 0, άρα αποτελείται από δύο ακμές, τις u-0 και 0-v. Το BEQ(4,21)= ακμή 0-4 (βάρος 1) καθώς min(w(0-4),w(0-21))=min(1,4)=1, ενώ το BEQ(1,24)= ακμή 0-1 (βάρος 5) καθώς min(w(0-1),w(0-24))=min(5,8)=5.

Στο tree4 (πλήρες δυαδικό δέντρο), τα εξεταζόμενα ερωτήματα που αφορούν κόμβους σε διαφορετικούς κλάδους επιστρέφουν συχνά ακμές με μικρό βάρος κοντά στη ρίζα, καθώς τα μονοπάτια μεταξύ κόμβων σε διαφορετικούς κλάδους διέρχονται από ακμές υψηλότερων επιπέδων. Για παράδειγμα, το BEQ(0,62)= ακμή 0-2 (βάρος 1), καθώς η ακμή αυτή ανήκει στο μοναδικό μονοπάτι από τον κόμβο 0 προς τον κόμβο 62 και έχει το μικρότερο βάρος πάνω σε αυτό το μονοπάτι. Αντίστοιχα, το BEQ(2,61) επιστρέφει την ακμή 2-6 με βάρος 5.

Συνολικά εκτελέστηκαν 38 ερωτήματα BEQ. Σε όλα τα ερωτήματα, τα αποτελέσματα της μεθόδου TreeQuery και της offline μεθόδου που βασίζεται στον Tarjan Offline LCA συμφώνησαν με τα αποτελέσματα της λύσης αναφοράς NaiveBEQ.

6.6 Πειράματα σε τυχαίους πίνακες μεγάλης κλίμακας

Τα προηγούμενα πειράματα του κεφαλαίου βασίζονται κυρίως σε σχετικά μικρούς και ελεγχόμενους πίνακες, οι οποίοι έχουν επιλεγεί έτσι ώστε να εξετάζονται διαφορετικές μορφές εισόδου, όπως αύξουσες και φθίνουσες ακολουθίες, πίνακες με διαφορετική κατανομή τιμών, καθώς και περιπτώσεις στις οποίες εμφανίζονται ίσες τιμές. Η χρήση τέτοιων παραδειγμάτων είναι σημαντική, καθώς μας επιτρέπει να παρατηρούμε άμεσα τη μορφή του Cartesian Tree και να ελέγχουμε ευκολότερα τη λειτουργία των διαφορετικών μεθόδων επίλυσης του προβλήματος RMQ.

Ωστόσο, τα μικρά και προκαθορισμένα παραδείγματα δεν επαρκούν από μόνα τους για να εξετάσουμε τη συμπεριφορά της υλοποίησης όταν το μέγεθος του πίνακα αυξάνεται σημαντικά. Για τον λόγο αυτό πραγματοποιούμε μία επιπλέον σειρά πειραμάτων, στην οποία χρησιμοποιούμε μεγαλύτερους πίνακες με τυχαία παραγόμενες τιμές.

Ο βασικός στόχος των συγκεκριμένων πειραμάτων είναι να εξετάσουμε αν οι διαφορετικές υλοποιήσεις που χρησιμοποιούμε για την απάντηση ερωτημάτων RMQ εξακολουθούν να επιστρέφουν τα ίδια αποτελέσματα όταν αυξάνεται σημαντικά το μέγεθος της εισόδου. Συγκεκριμένα, συγκρίνουμε τη μέθοδο NaiveRMQ, τη SparseTable, τη μέθοδο που βασίζεται στο Cartesian Tree και στην απλή επίλυση LCA, την OptimalLCA και την offline μέθοδο που βασίζεται στον Tarjan Offline LCA.

Για τον σκοπό αυτό επιλέξαμε τρία διαφορετικά μεγέθη εισόδου: 100, 1000 και 10000 στοιχεία. Κάθε νέο σύνολο δεδομένων είναι κατά μία τάξη μεγέθους μεγαλύτερο από το προηγούμενο. Με αυτόν τον τρόπο μπορούμε να εξετάσουμε τη συμπεριφορά και τη συμφωνία των υλοποιήσεων σε εισόδους αρκετά μεγαλύτερες από εκείνες των αρχικών δοκιμών.

Παραγωγή των τυχαίων πινάκων

Για τη δημιουργία των νέων συνόλων δεδομένων χρησιμοποιήσαμε μία απλή γεννήτρια τυχαίων πινάκων. Για κάθε πίνακα καθορίζουμε αρχικά το πλήθος n των στοιχείων του και στη συνέχεια παράγουμε n τυχαίους θετικούς ακέραιους αριθμούς.

Οι τιμές επιλέγονται τυχαία από το διάστημα 1 έως 100000. Επιλέξαμε ένα αρκετά μεγάλο εύρος τιμών ώστε οι πίνακες να μην ακολουθούν κάποιο συγκεκριμένο ή εύκολα προβλέψιμο μοτίβο. Σε αντίθεση με τους προηγούμενους χειροποίητους πίνακες, στους οποίους η διάταξη των στοιχείων έχει επιλεγεί σκόπιμα ώστε να εξετάσουμε συγκεκριμένες περιπτώσεις, στα νέα σύνολα δεδομένων οι τιμές δεν ακολουθούν κάποια προκαθορισμένη σειρά.

Η επιλογή τυχαίων δεδομένων μάς επιτρέπει επομένως να ελέγξουμε την υλοποίηση σε πιο γενικές εισόδους. Δεν προσπαθούμε με τα συγκεκριμένα πειράματα να δημιουργήσουμε τη χειρότερη δυνατή περίπτωση για κάποιον από τους αλγορίθμους, αλλά να εξετάσουμε αν οι διαφορετικές μέθοδοι εξακολουθούν να συμφωνούν όταν εφαρμόζονται σε μεγαλύτερους πίνακες χωρίς ειδικά σχεδιασμένη δομή.

Δεν χρησιμοποιήσαμε σταθερό seed στη γεννήτρια τυχαίων αριθμών. Τα αρχεία δημιουργήθηκαν μία φορά και στη συνέχεια τα διατηρήσαμε ως σταθερά αρχεία εισόδου. Επομένως, παρόλο που οι τιμές τους δημιουργήθηκαν αρχικά με τυχαίο τρόπο, όλες οι μεταγενέστερες εκτελέσεις των πειραμάτων πραγματοποιούνται πάνω στα ίδια ακριβώς δεδομένα.

Οι τρεις νέοι πίνακες που δημιουργήσαμε παρουσιάζονται στον ακόλουθο πίνακα:

Πίνακας	Μέγεθος	Αριθμός ερωτημάτων RMQ
array6	100	20
array7	1000	20
array8	10000	20

Πίνακας 6.5. Τυχαίοι πίνακες μεγάλης κλίμακας

Για τους μικρότερους πίνακες μπορούμε να εμφανίσουμε αναλυτικά τον αρχικό πίνακα, το Cartesian Tree, τη δομή που χρησιμοποιείται για το LCA και το Sparse Table. Στους μεγαλύτερους πίνακες, όμως, μία πλήρης εκτύπωση χιλιάδων στοιχείων και κόμβων θα δημιουργούσε πολύ μεγάλο όγκο εξόδου χωρίς να προσφέρει ουσιαστική επιπλέον πληροφορία.

Για τον λόγο αυτό, στους πίνακες των 1000 και 10000 στοιχείων παραλείπουμε την αναλυτική εκτύπωση των εσωτερικών δομών. Η κατασκευή του Cartesian Tree, των απαραίτητων δομών για τις μεθόδους LCA και του Sparse Table πραγματοποιείται κανονικά, αλλά το πρόγραμμα παραλείπει την αναλυτική εμφάνισή τους για τις μεγαλύτερες εισόδους. Με αυτόν τον τρόπο περιορίζουμε την έξοδο του προγράμματος και επικεντρωνόμαστε στα αποτελέσματα των RMQ ερωτημάτων, τα οποία αποτελούν το βασικό αντικείμενο του συγκεκριμένου ελέγχου.

Παραγωγή των RMQ ερωτημάτων

Για κάθε έναν από τους τρεις νέους πίνακες δημιουργήσαμε 20 τυχαία ερωτήματα RMQ. Κάθε ερώτημα έχει τη μορφή RMQ(l,r), όπου l και r είναι δύο δείκτες του πίνακα.

Οι δύο δείκτες επιλέγονται τυχαία, με τον περιορισμό l<r. Με αυτόν τον τρόπο εξασφαλίζουμε ότι κάθε ερώτημα αφορά ένα έγκυρο διάστημα του πίνακα και περιλαμβάνει περισσότερα από ένα στοιχεία. Αν οι δύο αρχικά επιλεγμένοι δείκτες εμφανιστούν με αντίστροφη σειρά, χρησιμοποιούμε τον μικρότερο ως αριστερό άκρο και τον μεγαλύτερο ως δεξί άκρο.

Ο αριθμός των 20 ερωτημάτων ανά σύνολο δεδομένων επιλέχθηκε ώστε να εξετάσουμε αρκετά διαφορετικά υποδιαστήματα κάθε πίνακα χωρίς να δημιουργούμε υπερβολικά μεγάλο όγκο αποτελεσμάτων. Ο στόχος δεν είναι να ελέγξουμε εξαντλητικά όλα τα πιθανά ερωτήματα. Για έναν πίνακα 10000 στοιχείων, άλλωστε, ο αριθμός των δυνατών διαστημάτων είναι πολύ μεγάλος. Αντίθετα, επιδιώκουμε έναν πρακτικό έλεγχο σε διαφορετικά τυχαία διαστήματα διαφορετικού μήκους και διαφορετικής θέσης μέσα στον πίνακα.

Συνολικά, για τους τρεις νέους πίνακες δημιουργήσαμε 60 τυχαία RMQ ερωτήματα.

Έλεγχος συμφωνίας των αποτελεσμάτων

Για κάθε ερώτημα RMQ υπολογίζουμε το αποτέλεσμα με πέντε διαφορετικούς τρόπους. Η πρώτη μέθοδος είναι η NaiveRMQ. Η συγκεκριμένη υλοποίηση εξετάζει διαδοχικά όλα τα στοιχεία που βρίσκονται μεταξύ των δεικτών l και r και διατηρεί τη μικρότερη τιμή που συναντά. Η μέθοδος αυτή είναι απλή και δεν απαιτεί κάποια ιδιαίτερη δομή προεπεξεργασίας. Για τον λόγο αυτό τη χρησιμοποιούμε ως βασική μέθοδο αναφοράς για τον έλεγχο των αποτελεσμάτων.

Η δεύτερη μέθοδος είναι η SparseTable. Πριν από την εκτέλεση των ερωτημάτων κατασκευάζεται ένας πίνακας προεπεξεργασμένων ελαχίστων για διαστήματα των οποίων το μήκος είναι δύναμη του 2. Με τη βοήθεια αυτής της πληροφορίας, ένα ερώτημα RMQ μπορεί να απαντηθεί χωρίς να χρειάζεται να εξεταστούν ένα προς ένα όλα τα στοιχεία του αντίστοιχου διαστήματος.

Η τρίτη μέθοδος βασίζεται στο Cartesian Tree και στην απλή μέθοδο LCA. Για ένα ερώτημα RMQ(l,r), οι δείκτες l και r αντιστοιχούν σε δύο κόμβους του Cartesian Tree και ο LCA τους αντιστοιχεί στο στοιχείο ελάχιστης τιμής του διαστήματος [l,r].

Η τέταρτη μέθοδος χρησιμοποιεί την OptimalLCA, η οποία βασίζεται σε Euler tour του Cartesian Tree και στην αναγωγή του LCA σε RMQ πάνω στον πίνακα των βαθών. Με την κατάλληλη προεπεξεργασία, η θεωρητική μέθοδος υποστηρίζει ερωτήματα LCA σε σταθερό χρόνο.

Η πέμπτη μέθοδος είναι η offline προσέγγιση με Tarjan LCA. Στην περίπτωση αυτή όλα τα ερωτήματα είναι γνωστά εκ των προτέρων και υπολογίζονται από κοινού πάνω στο Cartesian Tree με χρήση Union-Find.

Η σύγκριση των πέντε μεθόδων είναι χρήσιμη, επειδή χρησιμοποιούνται διαφορετικοί τρόποι υπολογισμού του ίδιου αποτελέσματος. Η NaiveRMQ εξετάζει απευθείας τα στοιχεία του αρχικού πίνακα, η SparseTable χρησιμοποιεί προϋπολογισμένες πληροφορίες, ενώ οι υπόλοιπες μέθοδοι αξιοποιούν τη σχέση μεταξύ RMQ και LCA μέσω του Cartesian Tree, είτε με online είτε με offline επεξεργασία.

Για κάθε ερώτημα το πρόγραμμα υπολογίζει το αποτέλεσμα με τις πέντε μεθόδους και ελέγχει αν οι τιμές τους είναι ίδιες. Όταν όλες οι τιμές συμφωνούν, εμφανίζεται η ένδειξη Match: YES.

Συνολικά, επομένως, πραγματοποιήσαμε 60 τυχαία RMQ ερωτήματα στους τρεις μεγαλύτερους πίνακες και σε όλα τα εξεταζόμενα ερωτήματα οι πέντε μέθοδοι συμφώνησαν.

Το αποτέλεσμα αυτό δεν αποτελεί τυπική απόδειξη ορθότητας των υλοποιήσεων, καθώς δεν εξετάζονται όλα τα δυνατά σύνολα δεδομένων και όλα τα δυνατά ερωτήματα. Αποτελεί όμως έναν σημαντικό πειραματικό έλεγχο, επειδή δείχνει ότι οι διαφορετικές υλοποιήσεις εξακολουθούν να παράγουν συνεπή αποτελέσματα όταν το μέγεθος της εισόδου αυξάνεται από 100 σε 1000 και τελικά σε 10000 στοιχεία.

Παράλληλα, τα πειράματα παρέχουν επιπλέον πειραματική ένδειξη για τη βασική σχέση μεταξύ RMQ και LCA. Παρόλο που οι διαφορετικές μέθοδοι ακολουθούν διαφορετικές διαδικασίες υπολογισμού, καταλήγουν στην ίδια ελάχιστη τιμή για όλα τα εξεταζόμενα διαστήματα. Με αυτόν τον τρόπο, τα πειράματα μεγάλης κλίμακας συμπληρώνουν τα μικρότερα ελεγχόμενα παραδείγματα και μας επιτρέπουν να ελέγξουμε τη λειτουργία των υλοποιήσεων σε μεγαλύτερες και περισσότερο γενικές εισόδους.

6.7 Πειράματα σε τυχαία δέντρα μεγάλης κλίμακας

Τα προηγούμενα πειράματα του κεφαλαίου βασίζονται κυρίως σε σχετικά μικρά και ελεγχόμενα δέντρα, τα οποία έχουν επιλεγεί έτσι ώστε να εξετάζονται συγκεκριμένες τοπολογίες, όπως ένα γραμμικό δέντρο, ένα δέντρο τύπου αστέρα ή ένα πλήρες δυαδικό δέντρο. Η χρήση τέτοιων παραδειγμάτων είναι σημαντική, καθώς επιτρέπει την άμεση παρατήρηση της μορφής του αρχικού δέντρου και του αντίστοιχου Cartesian Tree of Tree και διευκολύνει την κατανόηση της λειτουργίας των αλγορίθμων.

Ωστόσο, τα μικρά και προκαθορισμένα παραδείγματα δεν επαρκούν από μόνα τους για την αξιολόγηση της συμπεριφοράς της υλοποίησης όταν το μέγεθος της εισόδου αυξάνεται σημαντικά. Για τον λόγο αυτό πραγματοποιούμε μια δεύτερη σειρά πειραμάτων, στην οποία χρησιμοποιούμε μεγαλύτερα, τυχαία παραγόμενα σταθμισμένα δέντρα.

Ο βασικός στόχος αυτών των πειραμάτων είναι διπλός. Αρχικά, επιδιώκουμε να εξετάσουμε αν οι διαφορετικές υλοποιήσεις εξακολουθούν να παράγουν συνεπή αποτελέσματα σε εισόδους σημαντικά μεγαλύτερες από εκείνες των προηγούμενων δοκιμών. Παράλληλα, εξετάζουμε τη λειτουργία της κατασκευής του Cartesian Tree of Tree καθώς αυξάνεται το μέγεθος του αρχικού δέντρου.

Για τον σκοπό αυτό επιλέξαμε τρία διαφορετικά μεγέθη εισόδου: 100, 1000 και 10000 κόμβοι. Η επιλογή αυτών των τιμών έγινε ώστε κάθε νέο σύνολο δεδομένων να είναι κατά μία τάξη μεγέθους μεγαλύτερη από το προηγούμενο. Με αυτόν τον τρόπο είναι δυνατό να παρατηρήσουμε πιο καθαρά πώς μεταβάλλεται η πρακτική συμπεριφορά της υλοποίησης όταν ο αριθμός των κόμβων αυξάνεται σημαντικά.

Παραγωγή των τυχαίων δέντρων

Για τη δημιουργία των νέων συνόλων δεδομένων χρησιμοποιήσαμε μία απλή γεννήτρια τυχαίων δέντρων. Η παραγωγή κάθε δέντρου πραγματοποιείται σταδιακά. Θεωρούμε τον κόμβο 0 ως τον αρχικό κόμβο και, για κάθε νέο κόμβο i, με i>0, επιλέγουμε τυχαία έναν κόμβο από το σύνολο {0,1, ... , i-1} και δημιουργούμε μία ακμή μεταξύ του νέου κόμβου και του επιλεγμένου προηγούμενου κόμβου.

Η επιλογή αυτής της διαδικασίας δεν είναι αυθαίρετη. Καθώς κάθε νέος κόμβος συνδέεται με ακριβώς έναν κόμβο που έχει ήδη ενταχθεί στο δέντρο, ο νέος κόμβος συνδέεται πάντοτε με το ήδη υπάρχον συνεκτικό τμήμα της δομής. Επομένως, μετά την ολοκλήρωση της διαδικασίας, όλοι οι κόμβοι ανήκουν στο ίδιο συνεκτικό γράφημα.

Παράλληλα, η προσθήκη κάθε νέου κόμβου με ακριβώς μία νέα ακμή δεν μπορεί να δημιουργήσει κύκλο, επειδή ο νέος κόμβος δεν υπήρχε προηγουμένως στη δομή. Για n κόμβους δημιουργούνται συνολικά n-1 ακμές. Συνεπώς, το παραγόμενο γράφημα είναι συνεκτικό και άκυκλο και επομένως αποτελεί δέντρο.

Σε κάθε ακμή ανατίθεται ένα τυχαίο θετικό βάρος από 1 έως 1000. Η χρήση μεγαλύτερου εύρους βαρών, σε σχέση με ορισμένα από τα μικρά χειροποίητα παραδείγματα, επιτρέπει τη δημιουργία πιο γενικών στιγμιοτύπων, στα οποία τα ελάχιστα βάρη των διαφορετικών μονοπατιών δεν ακολουθούν κάποιο προκαθορισμένο μοτίβο.

Δεν χρησιμοποιήσαμε κάποιο σταθερό seed στη γεννήτρια τυχαίων αριθμών. Τα αρχεία τα δημιουργήσαμε μία φορά και στη συνέχεια τα διατηρήσαμε ως σταθερά αρχεία εισόδου για τα πειράματα. Με τον τρόπο αυτόν, παρόλο που η αρχική τους δημιουργία ήταν τυχαία, όλες οι μεταγενέστερες εκτελέσεις των πειραμάτων χρησιμοποιούν τα ίδια ακριβώς δεδομένα.

Τα τρία δέντρα που προέκυψαν έχουν τα ακόλουθα χαρακτηριστικά:

Δέντρο	Κόμβοι	Ακμές	Ύψος αρχικού δέντρου
tree5	100	99	8
tree6	1000	999	13
tree7	10000	9999	16

Πίνακας 6.6. Τυχαία δέντρα μεγάλης κλίμακας

Το ύψος το υπολογίσαμε κατά τη διάρκεια της παραγωγής θεωρώντας τον κόμβο 0 ως ρίζα. Παρατηρήσαμε ότι, παρόλο που ο αριθμός των κόμβων αυξάνεται κατά μία τάξη μεγέθους κάθε φορά, το ύψος των συγκεκριμένων τυχαίων δέντρων παραμένει

σχετικά μικρό. Αυτό διαφοροποιεί τα συγκεκριμένα πειράματα από ακραίες περιπτώσεις, όπως ένα γραμμικό δέντρο, όπου το ύψος είναι n-1.

Επομένως, τα νέα πειράματα δεν έχουν στόχο να δημιουργήσουν τη χειρότερη δυνατή τοπολογία. Αντίθετα, εξετάζουν τη συμπεριφορά της υλοποίησης σε μεγαλύτερες, τυχαίες και περισσότερο γενικές εισόδους.

Παραγωγή των BEQ ερωτημάτων

Για κάθε ένα από τα τρία νέα δέντρα δημιουργήσαμε 20 τυχαία ερωτήματα BEQ. Κάθε ερώτημα έχει τη μορφή BEQ(u,v), όπου u και v είναι δύο διαφορετικοί κόμβοι του αρχικού δέντρου. Οι δύο κόμβοι επιλέγονται τυχαία από το σύνολο των κόμβων του αντίστοιχου συνόλου δεδομένων, με τον περιορισμό u≠v. Ο περιορισμός αυτός χρησιμοποιείται επειδή ένα ερώτημα μεταξύ ενός κόμβου και του εαυτού του δεν περιλαμβάνει κάποια ακμή και συνεπώς δεν είναι χρήσιμο για τον έλεγχο της λειτουργίας του BEQ.

Ο αριθμός των 20 ερωτημάτων ανά σύνολο δεδομένων επιλέχθηκε ώστε να υπάρχει ένα επαρκές σύνολο διαφορετικών ζευγών κόμβων χωρίς να παράγεται υπερβολικά μεγάλο αποτέλεσμα στην έξοδο του προγράμματος. Ο σκοπός των συγκεκριμένων ερωτημάτων δεν είναι να εξετάσουμε εξαντλητικά όλα τα δυνατά ζεύγη κόμβων, αλλά να πραγματοποιήσουμε έναν πρακτικό έλεγχο της συμφωνίας των αποτελεσμάτων σε διαφορετικά τυχαία μονοπάτια μέσα σε μεγαλύτερα δέντρα.

Έλεγχος συμφωνίας των αποτελεσμάτων

Για κάθε ερώτημα BEQ υπολογίζουμε το αποτέλεσμα με τρεις διαφορετικούς τρόπους. Η πρώτη μέθοδος είναι η TreeQuery, η οποία χρησιμοποιεί το Cartesian Tree of Tree. Οι αρχικοί κόμβοι u και v αντιστοιχίζονται στα φύλλα του Cartesian Tree of Tree και στη συνέχεια υπολογίζεται ο LCA των δύο φύλλων. Η ακμή που αντιστοιχεί στον συγκεκριμένο εσωτερικό κόμβο αποτελεί την ακμή ελάχιστου βάρους στο μονοπάτι μεταξύ u και v στο αρχικό δέντρο.

Η δεύτερη μέθοδος είναι η offline προσέγγιση που βασίζεται στον Tarjan Offline LCA. Στην περίπτωση αυτή όλα τα ερωτήματα BEQ είναι γνωστά εκ των προτέρων και μετατρέπονται σε ερωτήματα LCA μεταξύ των αντίστοιχων φύλλων του Cartesian Tree of Tree. Τα ερωτήματα υπολογίζονται από κοινού με χρήση της δομής Union-Find και η ακμή που αντιστοιχεί στον LCA αποτελεί την απάντηση του αντίστοιχου ερωτήματος BEQ.

Η τρίτη μέθοδος είναι η NaiveBEQ. Η μέθοδος αυτή εντοπίζει άμεσα το μονοπάτι μεταξύ δύο κόμβων στο αρχικό δέντρο και εξετάζει τις ακμές του μονοπατιού προκειμένου να βρει εκείνη με το μικρότερο βάρος. Χρησιμοποιούμε τη NaiveBEQ ως μέθοδο αναφοράς, επειδή η λογική της είναι άμεση και ανεξάρτητη από το Cartesian Tree of Tree.

Για κάθε ερώτημα το πρόγραμμα συγκρίνει τις ακμές που επιστρέφουν οι τρεις μέθοδοι. Σε όλα τα 20 ερωτήματα κάθε ενός από τα τρία σύνολα δεδομένων, η TreeQuery, η offline μέθοδος και η NaiveBEQ επέστρεψαν τα ίδια αποτελέσματα.

Συνολικά, επομένως, πραγματοποιήθηκαν 60 τυχαία BEQ ερωτήματα στα τρία μεγαλύτερα δέντρα και σε όλα οι τρεις μέθοδοι συμφώνησαν. Το αποτέλεσμα αυτό δεν αποτελεί τυπική απόδειξη ορθότητας των αλγορίθμων. Αποτελεί, ωστόσο, ισχυρή πειραματική ένδειξη ότι οι διαφορετικές υλοποιήσεις παράγουν συνεπή αποτελέσματα και σε εισόδους σημαντικά μεγαλύτερες από τα αρχικά παραδείγματα.

6.8 Σύνοψη πειραματικών αποτελεσμάτων

Τα πειράματα που πραγματοποιήσαμε είχαν ως βασικό στόχο τον έλεγχο της ορθότητας των υλοποιήσεων για τα προβλήματα RMQ και BEQ. Αρχικά χρησιμοποιήσαμε μικρά και ελεγχόμενα παραδείγματα, ώστε να μπορούμε να παρατηρήσουμε άμεσα τη μορφή των δομών και τη λειτουργία των αλγορίθμων, και στη συνέχεια επεκτείναμε τις δοκιμές σε μεγαλύτερα τυχαία σύνολα δεδομένων.

Στην περίπτωση των πινάκων συγκρίναμε τη NaiveRMQ, τη SparseTable, την προσέγγιση Cartesian Tree + LCA, την OptimalLCA και την offline μέθοδο που βασίζεται στον Tarjan Offline LCA. Στους μεγάλους τυχαίους πίνακες των 100, 1000 και 10000 στοιχείων εκτελέσαμε συνολικά 60 τυχαία RMQ ερωτήματα και σε όλα οι πέντε μέθοδοι επέστρεψαν το ίδιο αποτέλεσμα.

Αντίστοιχα, στα σταθμισμένα δέντρα συγκρίναμε τη TreeQuery, η οποία βασίζεται στο Cartesian Tree of Tree και στον LCA, την offline μέθοδο που βασίζεται στον Tarjan Offline LCA και τη NaiveBEQ. Στα μεγάλα τυχαία δέντρα των 100, 1000 και 10000 κόμβων εκτελέσαμε συνολικά 60 τυχαία BEQ ερωτήματα και σε όλα οι τρεις μέθοδοι συμφώνησαν.

Τα αποτελέσματα αυτά δεν αποτελούν τυπική απόδειξη ορθότητας, αλλά παρέχουν ισχυρή πειραματική ένδειξη ότι οι διαφορετικές υλοποιήσεις παράγουν συνεπή αποτελέσματα στα σύνολα δεδομένων που εξετάστηκαν. Παράλληλα, παρέχουν πειραματική επιβεβαίωση της αναμενόμενης σχέσης μεταξύ RMQ και LCA μέσω του

Cartesian Tree, καθώς και της αντίστοιχης χρήσης του Cartesian Tree of Tree για την απάντηση ερωτημάτων BEQ.

Τέλος, τα πειράματα δείχνουν ότι οι διαφορετικές online και offline προσεγγίσεις που υλοποιήθηκαν συμφωνούν ως προς τα αποτελέσματά τους στα εξεταζόμενα σύνολα δεδομένων. Η πειραματική αξιολόγηση επικεντρώνεται κυρίως στον έλεγχο της ορθής και συνεπούς λειτουργίας των υλοποιήσεων και όχι σε πλήρη συγκριτική αξιολόγηση των χρόνων εκτέλεσης.

Κεφάλαιο 7. Συμπεράσματα

Στην παρούσα διπλωματική εργασία μελετήσαμε και υλοποιήσαμε δομές δεδομένων και αλγορίθμους για την απάντηση ερωτημάτων σε πίνακες και δέντρα, εστιάζοντας στα προβλήματα Range Minimum Query (RMQ), Lowest Common Ancestor (LCA) και Bottleneck Edge Query (BEQ).

7.1 Σύνοψη Αποτελεσμάτων

Για το πρόβλημα RMQ μελετήσαμε τη θεμελιώδη σχέση μεταξύ RMQ και LCA μέσω του Cartesian Tree. Κάθε ερώτημα RMQ(i,j) σε έναν πίνακα A μπορεί να μετατραπεί σε ερώτημα LCA μεταξύ των κόμβων που αντιστοιχούν στις θέσεις i και j στο Cartesian Tree του A, και ο κόμβος LCA αντιστοιχεί στο ελάχιστο στοιχείο του συγκεκριμένου διαστήματος. Το Cartesian Tree κατασκευάζεται στην υλοποίηση σε χρόνο O(n) με χρήση μονοτονικής στοίβας.

Για την επίλυση του RMQ υλοποιήθηκαν διαφορετικές προσεγγίσεις. Το Sparse Table απαιτεί O(n log n) προεπεξεργασία και απαντά κάθε ερώτημα σε O(1) χρόνο. Μέσω του Cartesian Tree χρησιμοποιήθηκε αρχικά μία απλή online μέθοδος LCA, ενώ υλοποιήθηκε επίσης η OptimalLCA, η οποία βασίζεται σε Euler tour και στην αναγωγή του LCA σε ειδική περίπτωση RMQ όπου οι διαδοχικές τιμές βάθους διαφέρουν κατά ±1. Η θεωρητική μέθοδος επιτυγχάνει γραμμική προεπεξεργασία και O(1) χρόνο ανά ερώτημα. Επιπλέον, υλοποιήθηκε offline επίλυση των ερωτημάτων LCA με τον αλγόριθμο του Tarjan και χρήση της δομής Union-Find. Ως απλή λύση αναφοράς χρησιμοποιήθηκε η NaiveRMQ.

Για το πρόβλημα BEQ μελετήσαμε το Cartesian Tree of Tree, μία αντίστοιχη δομή για σταθμισμένα δέντρα. Τα φύλλα του Cartesian Tree of Tree αντιστοιχούν στους κόμβους του αρχικού δέντρου, ενώ οι εσωτερικοί κόμβοι αντιστοιχούν στις ακμές του. Για δύο κόμβους u και v, η ακμή που αντιστοιχεί στον LCA των αντίστοιχων φύλλων είναι η ακμή ελάχιστου βάρους στο μοναδικό μονοπάτι μεταξύ u και v.

Η κατασκευή του Cartesian Tree of Tree υλοποιήθηκε με ταξινόμηση των ακμών κατά φθίνουσα σειρά βάρους και χρήση της δομής Union-Find για τη σταδιακή συνένωση των συνδεδεμένων συνιστωσών. Για ένα δέντρο με n κόμβους η κατασκευή απαιτεί O(n log n) χρόνο, με την ταξινόμηση των ακμών να αποτελεί το κυρίαρχο κόστος. Για την

απάντηση των BEQ ερωτημάτων χρησιμοποιήθηκε τόσο η online προσέγγιση μέσω LCA όσο και η offline προσέγγιση με τον αλγόριθμο Tarjan πάνω στο Cartesian Tree of Tree.

Στα ελεγχόμενα test cases εκτελέστηκαν συνολικά 65 ερωτήματα RMQ και 38 ερωτήματα BEQ, ενώ στα πειράματα μεγάλης κλίμακας εκτελέστηκαν επιπλέον 60 τυχαία ερωτήματα RMQ και 60 τυχαία ερωτήματα BEQ. Σε όλα τα εξεταζόμενα ερωτήματα, οι διαφορετικές μέθοδοι συμφώνησαν με τις αντίστοιχες λύσεις αναφοράς. Τα αποτελέσματα αυτά δεν αποτελούν τυπική απόδειξη ορθότητας, αλλά παρέχουν ισχυρή πειραματική ένδειξη για τη συνεπή λειτουργία των υλοποιήσεων στα σύνολα δεδομένων που εξετάστηκαν.

7.2 Συμπεράσματα

Από τη μελέτη και υλοποίηση των παραπάνω δομών δεδομένων και αλγορίθμων προκύπτουν τα ακόλουθα αποτελέσματα:

Πρώτον, η αναγωγή του RMQ σε LCA μέσω του Cartesian Tree αποτελεί μια σημαντική δομική σχέση, η οποία μετατρέπει ένα ερώτημα ελαχίστου σε ερώτημα πάνω σε δέντρο. Η σχέση αυτή επιτρέπει την αξιοποίηση αλγορίθμων LCA για την επίλυση του RMQ μέσω του Cartesian Tree.

Δεύτερον, το Sparse Table αποτελεί μία αποδοτική λύση για το RMQ, καθώς απαιτεί O(n log n) προεπεξεργασία και επιτρέπει την απάντηση κάθε ερωτήματος σε O(1) χρόνο. Παράλληλα, η προσέγγιση μέσω Cartesian Tree και LCA δείχνει ότι το ίδιο πρόβλημα μπορεί να αντιμετωπιστεί μέσω διαφορετικής αναγωγής. Η χρήση της OptimalLCA επιτρέπει θεωρητικά γραμμική προεπεξεργασία και σταθερό χρόνο ανά ερώτημα, ενώ η offline προσέγγιση του Tarjan επιτρέπει την κοινή επεξεργασία ενός συνόλου προκαθορισμένων ερωτημάτων με χρήση Union-Find.

Τρίτον, το Cartesian Tree μπορεί να κατασκευαστεί σε γραμμικό χρόνο με χρήση μονοτονικής στοίβας. Η ιδιότητα αυτή είναι σημαντική, καθώς εξαλείφει την εξάρτηση του χρόνου κατασκευής από το σχήμα του παραγόμενου δέντρου και επιτρέπει τη χρήση της δομής ακόμη και σε περιπτώσεις όπου το Cartesian Tree είναι έντονα μη ισορροπημένο.

Τέταρτον, το Cartesian Tree of Tree αποτελεί γενίκευση της ιδέας του Cartesian Tree από πίνακες σε σταθμισμένα δέντρα. Η σχέση μεταξύ BEQ και LCA επιτρέπει την απάντηση ενός ερωτήματος BEQ μέσω του LCA των αντίστοιχων φύλλων στο Cartesian Tree of Tree. Στην παρούσα υλοποίηση, η κατασκευή της δομής πραγματοποιείται με ταξινόμηση των ακμών κατά φθίνουσα σειρά βάρους και χρήση Union-Find, με συνολικό χρόνο O(n log n) για δέντρο με n κόμβους.

Πέμπτον, η χρήση online και offline μεθόδων LCA αναδεικνύει δύο διαφορετικούς τρόπους αντιμετώπισης των ερωτημάτων. Οι online μέθοδοι είναι κατάλληλες όταν τα ερωτήματα υποβάλλονται ανεξάρτητα μετά την προεπεξεργασία, ενώ η offline μέθοδος του Tarjan είναι ιδιαίτερα χρήσιμη όταν όλα τα ερωτήματα είναι γνωστά εκ των προτέρων και μπορούν να υπολογιστούν από κοινού.

Τέλος, η χρήση απλοϊκών λύσεων αναφοράς αποτέλεσε χρήσιμο εργαλείο για τον πειραματικό έλεγχο των πιο σύνθετων υλοποιήσεων. Οι NaiveRMQ και NaiveBEQ ακολουθούν άμεσες διαδικασίες υπολογισμού και χρησιμοποιήθηκαν ως ανεξάρτητες λύσεις αναφοράς. Η συμφωνία των αποτελεσμάτων τους με τις υπόλοιπες μεθόδους στα εξεταζόμενα test cases παρέχει ισχυρή πειραματική ένδειξη για τη συνεπή λειτουργία των υλοποιήσεων.

7.3 Προτάσεις για Περαιτέρω Έρευνα

Η παρούσα εργασία μπορεί να επεκταθεί προς διάφορες κατευθύνσεις, τόσο ως προς τη θεωρητική μελέτη όσο και ως προς την υλοποίηση και την πειραματική αξιολόγηση των αλγορίθμων.

Όσον αφορά το RMQ και το LCA, μία πιθανή επέκταση είναι η περαιτέρω βελτιστοποίηση της υλοποίησης της θεωρητικά βέλτιστης μεθόδου LCA. Στην παρούσα εργασία υλοποιήθηκε προσέγγιση που βασίζεται σε Euler tour και στην ειδική μορφή RMQ όπου οι διαδοχικές τιμές βάθους διαφέρουν κατά ±1. Η θεωρητική μέθοδος επιτυγχάνει γραμμικό χρόνο προεπεξεργασίας και σταθερό χρόνο ανά ερώτημα. Μία μελλοντική εργασία θα μπορούσε να εξετάσει εναλλακτικές υλοποιητικές παραλλαγές της ίδιας μεθόδου και να συγκρίνει αναλυτικότερα το κόστος προεπεξεργασίας, τη χρήση μνήμης και την πρακτική απόδοση των ερωτημάτων.

Επιπλέον, η offline μέθοδος του Tarjan θα μπορούσε να μελετηθεί σε μεγαλύτερα σύνολα προκαθορισμένων ερωτημάτων και να συγκριθεί συστηματικά με τις online μεθόδους LCA. Μία τέτοια μελέτη θα επέτρεπε να εξεταστεί σε ποιες περιπτώσεις είναι προτιμότερη η κοινή offline επεξεργασία των ερωτημάτων και σε ποιες είναι καταλληλότερη η χρήση μίας δομής που υποστηρίζει ανεξάρτητα online ερωτήματα.

Για το BEQ, μία φυσική κατεύθυνση περαιτέρω μελέτης είναι η υλοποίηση της θεωρητικά βέλτιστης κατασκευής του Cartesian Tree of Tree που παρουσιάζεται στη βιβλιογραφία. Στην παρούσα εργασία χρησιμοποιήθηκε μία απλούστερη και πρακτικότερη προσέγγιση, η οποία βασίζεται στην ταξινόμηση των ακμών κατά φθίνουσα σειρά βάρους και στη χρήση της δομής Union-Find για τη συνένωση των συνδεδεμένων συνιστωσών, με συνολικό χρόνο $O(n \log n)$. Η προσέγγιση αυτή βελτιώνει

την αρχική αναδρομική υλοποίηση, αλλά δεν υλοποιεί τη θεωρητικά βέλτιστη μέθοδο που περιγράφεται από τους Demaine, Landau και Weimann. Η μέθοδος που παρουσιάζεται στην εργασία των Demaine, Landau και Weimann βασίζεται σε πιο σύνθετες τεχνικές διαχείρισης συνδεσιμότητας και επιτυγχάνει καλύτερο θεωρητικό όριο κατασκευής. Η υλοποίηση και πειραματική αξιολόγηση αυτής της προσέγγισης αποτελεί επομένως φυσική κατεύθυνση μελλοντικής επέκτασης.

Μία επιπλέον κατεύθυνση μελλοντικής έρευνας είναι η περαιτέρω μελέτη των online και offline μεθόδων για την απάντηση ερωτημάτων BEQ πάνω στο Cartesian Tree of Tree. Η offline προσέγγιση με Tarjan επιτρέπει την κοινή επεξεργασία ενός συνόλου προκαθορισμένων ερωτημάτων, ενώ μία κατάλληλα προεπεξεργασμένη online δομή LCA μπορεί να χρησιμοποιηθεί όταν τα ερωτήματα δεν είναι γνωστά εκ των προτέρων. Η σύγκριση των δύο προσεγγίσεων σε διαφορετικά μεγέθη και πλήθη ερωτημάτων θα μπορούσε να δώσει μία πληρέστερη εικόνα της πρακτικής τους συμπεριφοράς.

Βιβλιογραφία

[1] E. D. Demaine, G. M. Landau, O. Weimann, “On Cartesian Trees and Range Minimum Queries,” *Algorithmica*, vol. 68, no. 3, pp. 610–625, 2014. DOI: 10.1007/s00453-012-9683-x.

[2] H. N. Gabow, J. L. Bentley, R. E. Tarjan, “Scaling and Related Techniques for Geometry Problems,” *Proceedings of the 16th ACM Symposium on Theory of Computing (STOC)*, pp. 135–143, 1984. DOI: 10.1145/800057.808675.

[3] D. Harel, R. E. Tarjan, “Fast Algorithms for Finding Nearest Common Ancestors,” *SIAM Journal on Computing*, vol. 13, no. 2, pp. 338–355, 1984. DOI: 10.1137/0213024.

[4] M. A. Bender, M. Farach-Colton, G. Pemmasani, S. Skiena, P. Sumazin, “Lowest Common Ancestors in Trees and Directed Acyclic Graphs,” *Journal of Algorithms*, vol. 57, no. 2, pp. 75–94, 2005. DOI: 10.1016/j.jalgor.2005.08.001.

[5] A. V. Aho, J. E. Hopcroft, J. D. Ullman, “On Finding Lowest Common Ancestors in Trees,” *SIAM Journal on Computing*, vol. 5, no. 1, pp. 115–132, 1976. DOI: 10.1137/0205011.

[6] R. E. Tarjan, “Applications of Path Compression on Balanced Trees,” *Journal of the ACM*, vol. 26, no. 4, pp. 690–715, 1979. DOI: 10.1145/322154.322161.

[7] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.

[8] CP-Algorithms, “Sparse Table.”

[9] CP-Algorithms, “Solve RMQ by finding LCA.”

[10] CP-Algorithms, “Lowest Common Ancestor – Tarjan’s Off-line Algorithm.”